

Design and Performance Analysis of an Intelligent Sensor and Locally Hosted Web-Based IoT Framework for Integrated Aquaponics Farming: A Case Study for the Sub-Saharan Region of Africa

by

Alusine Bah (210021165)
Abdoulatuf Mhadjou (210021153)
Bello Aminu Ado (210021157)

**BACHELOR OF SCIENCE
IN
ELECTRICAL AND ELECTRONIC ENGINEERING**



Department of Electrical and Electronic Engineering
Islamic University of Technology (IUT)
Board Bazar, Gazipur-1704, Bangladesh.

September 2026

DECLARATION

It is hereby declared that this thesis report is only submitted to the Electrical and Electronic Engineering Department. Any part of it has not been submitted elsewhere for the award of any Degree or Diploma.

Alusine Bah
210021165

Abdoulatuf Mhadjou
210021153

Bello Aminu Ado
210021157

APPROVAL PAGE

Design and Performance Analysis of an Intelligent Sensor and Locally Hosted Web-Based IoT Framework for Integrated Aquaponics Farming: A Case Study for the Sub-Saharan Region of Africa

Submitted by:

Alusine Bah (210021165)
Abdoulatuf Mhadjou (210021153)
Bello Aminu Ado (210021157)

The thesis has been found as satisfactory and accepted as partial fulfillment of the requirement for the Degree of BACHELOR OF SCIENCE IN ELECTRICAL AND ELECTRONIC ENGINEERING of Islamic University of Technology (IUT).

Prof. Dr. Khondokar Habibul Kabir

Professor,

Department of Electrical and Electronic Engineering (EEE),
Islamic University of Technology (IUT),
Board Bazar, Gazipur-1704, Bangladesh

Date of Approval: _____

Dedicated to our families.

ACKNOWLEDGMENTS

First and foremost, we express our gratitude to Allah for granting us health and strength to navigate through this journey.

We would like to express our deepest gratitude to our thesis supervisor, Prof. Dr. Khondokar Habibul Kabir, for his invaluable guidance and support throughout this research.

We are also grateful to our colleagues, peers, and the Department of Electrical and Electronic Engineering for providing a supportive research environment. A special thanks to our parents for their unwavering support; their encouragement has been our pillar of strength.

ABSTRACT

Food insecurity in Sub-Saharan Africa is worsened by reliance on traditional farming, which is sensitive to soil, rainfall and manual oversight. Aquaponics combines fish farming with soilless crop growth in a closed loop, but its biological complexity demands continuous monitoring and automatic control. This thesis presents the design, construction and evaluation of an intelligent sensor-based IoT framework for integrated aquaponics farming, built on a hybrid Arduino Mega and ESP32 architecture. The Mega performs all control decisions locally using threshold-based hysteresis, while the ESP32 hosts the Smart Aquaponics Operating System (SAOS) dashboard for monitoring, manual override, sensor diagnostics and REST access. The prototype is solar-powered and was tested with live fish and plants across seventeen measured parameters and ten actuators. Performance was evaluated in terms of communication latency, packet delivery ratio, system response time and energy consumption. The mean latency was 2 ms, the packet delivery ratio was 93 %, and the mean response time was 0.029 s, with a nominal draw of 17.2 W and a peak of 26.9 W. The results show that a reliable, locally controlled aquaponics system can be built from commercially available components and is a practical option for small-scale farming in resource-limited regions.

Keywords: Aquaponics, Internet of Things, Smart Aquaponics Operating System, Solar-Powered Automation, Hybrid Controller Architecture

Contents

DECLARATION	ii
APPROVAL PAGE	iii
ACKNOWLEDGMENTS	v
ABSTRACT	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Background and Historical Context	2
1.3 Problem Statement: Non-Technical Perspective	3
1.3.1 Limitations of Existing Work	4
1.3.2 Motivation of the Project	6
1.4 Research Gaps: Technical Perspective	6
1.5 Research Questions	7
1.6 Problem Identification: Technical Statement	7
1.7 Research Aim and Objectives	8
1.8 Expected Outcomes	8
1.9 Methodology: Technical Blueprint	9
1.10 Novelty, Contribution, and Importance of the Work	9
1.11 Significance and Rationale	10
1.12 Organization of the Thesis	11
2 LITERATURE REVIEW	13
2.1 Overview	13
2.2 Related Work	13
2.2.1 Aquaponics Farming Systems and Operational Monitoring Requirements	13
2.2.2 IoT-Based Monitoring and Control in Aquaponics	14
2.2.3 Wireless Communication Technologies	14
2.2.4 Cloud-Based Data Management	15
2.3 Limitations of Existing Work	16
2.4 Research Contribution: Addressing and Resolving the Existing Research Gap .	16

3	METHODOLOGY	19
3.1	Overview	19
3.2	Dataset / Materials	19
3.3	Baseline / Initial Approach	21
3.4	Evaluation Metrics	23
3.4.1	Communication Latency	23
3.4.2	Packet Delivery Ratio	23
3.4.3	System Response Time	24
3.4.4	Energy Consumption	24
3.4.5	Test Plan	25
4	PROPOSED SYSTEM / MODEL	26
4.1	Motivation	26
4.2	System / Architecture Overview	26
4.3	Detailed Design	28
4.3.1	Sensing and Actuation Layer	28
4.3.2	Local Control Layer	28
4.3.3	Local Link Layer	29
4.3.4	Gateway and Hosting Layer	30
4.3.5	Power Subsystem	30
4.4	Implementation Details	30
5	IMPLEMENTATION	32
5.1	Overview	32
5.2	Design / Approach	32
5.3	Integration	35
5.3.1	Mega Firmware	35
5.3.2	ESP32 Firmware	36
5.3.3	SAOS Dashboard	36
5.3.4	MATLAB/Simulink Logic Verification	37
5.4	Experimental Setup	39
6	RESULTS AND DISCUSSION	40
6.1	Results and Evaluation	40
6.2	Discussion	42
6.3	Limitations	43
6.4	Comparison with Literature	43

6.5	Qualitative Results	44
6.6	Experimental Setup	46
7	CONCLUSION AND FUTURE WORK	47
7.1	Conclusion	47
7.2	Limitations	48
7.3	Future Work	48
8	DEMONSTRATION OF OUTCOME BASED EDUCATION (OBE)	50
8.1	Introduction	50
8.2	Program Outcomes (PO) Addressed	50
8.2.1	Program-Outcome Justification	51
8.3	Complex Engineering Problem (CEP) Attributes	52
8.3.1	P1–P7 Attribute Mapping	53
8.3.2	Socio-Cultural, Environmental and Ethical Impact	55
8.4	Complex Engineering Activity (CEA) Attributes	55
8.4.1	A1–A5 Attribute Mapping	56
8.5	Mapping of Objectives to Outcomes	56
8.5.1	Course Outcomes (COs) and Program Outcomes (POs)	57
8.5.2	Aspects of Program Outcomes Addressed	57
8.5.3	Knowledge Profiles (K3–K8) Addressed	59
8.5.4	OBE Summary	60
	REFERENCES	61
A	Source Code	65
A.1	Mega Firmware	65
A.1.1	Safe Startup Sequence	65
A.1.2	Main Control Loop	66
A.1.3	Telemetry Packet Builder	67
A.2	ESP32 Firmware	68
A.2.1	Initialization	68
A.2.2	Main Service Loop	69
A.2.3	UART Manager Interface	70
A.2.4	Web Manager Interface	71
A.2.5	System Entry Points	72
A.3	Supporting Modules	72

B	Additional Results	74
B.1	Communication Latency Summary	74
B.2	Packet Delivery Ratio Summary	75
B.3	System Response Time Trials	75
B.4	Component Power Rating Table	76
B.5	Availability of Raw Logs	77
C	Pseudocode	78
C.1	General Two-Threshold Hysteresis Law	78
C.2	Application to the Circulation Pump	78
C.3	Relationship to the Simulink Model	79

List of Figures

3.1	Proteus schematic of aquaponics prototype.	20
3.2	moisture thresholds	22
4.1	Five-layer system architecture.	27
5.1	Photograph of the vertical aquaponics prototype.	35
5.2	MATLAB/Simulink model of the hysteresis control law.	37
5.3	Pump control signal output.	38
6.1	SAOS dashboard.	45

List of Tables

2.1	Comparison of wireless communication technologies.	18
3.1	Measured parameters and their sensor channels (17 parameters).	20
3.2	Ten actuators controlled by the Mega 2560 through the relay bank.	21
5.1	Rated power for each prototype component.	34
6.1	Measured results against literature benchmarks.	41
6.2	Comparison of the proposed system against related IoT aquaponics systems. . .	41
8.1	Program Outcomes (POs) addressed by the completed thesis.	51
8.2	Complex Engineering Problem (CEP) attributes addressed.	54
8.3	Complex Engineering Activity (CEA) attributes addressed.	56
8.4	Mapping of thesis objectives to Program Outcomes.	57
8.5	Course-outcome mapping from pre-defence to completed thesis.	58
8.6	Selected aspects of Program Outcomes addressed by the thesis.	59
8.7	Knowledge Profiles (K3–K8) addressed by the thesis.	60
B.1	Communication latency summary over $N = 2000$ packets.	74
B.2	Packet delivery ratio summary over a 30-minute window at an effective transmit rate of approximately 0.55 frames per second.	75
B.3	Individual system-response-time trials.	75
B.4	Component power ratings for the prototype. All values are manufacturer-rated, not bench-measured.	76

Chapter 1

INTRODUCTION

1.1 Introduction

Communities in Sub-Saharan Africa (SSA) are mainly engaged in agriculture, but they are experiencing a growing food security crisis [1]. The United States Department of Agriculture states that in 2024, food insecurity was a concern for 29.3% of the population of SSA, and the region has the lowest average annual per capita GDP of any of the areas examined worldwide, at \$1,387 [2]. The 2022 FAO report reported that there were about 239.6 million malnourished individuals in the region, with the Eastern Africa region alone accounting for 135 million undernourished individuals [1]. These numbers are indicative of a food system under extreme stress, one in which traditional farming that relies on rainfall, productive soil and manual labour is not keeping up with population growth and the speed at which the environment is changing.

Aquaponics, a closed-loop integrated agriculture system that combines fish farming with soilless plant growth, has proved to be an effective and resource-efficient option [3]. It produces both animal protein and fresh vegetables in a very small area, uses up to 90 per cent less water than traditional soil-based farming, produces no chemical runoff, and does not depend on soil quality. For SSA, these characteristics make aquaponics directly relevant. However, the biological complexity of aquaponics requires continuous monitoring and precise automated control of multiple environmental parameters. This requirement far exceeds the capabilities of manual observation. The Internet of Things (IoT) provides the technical infrastructure to meet this requirement through embedded sensor networks, wireless communication and network-enabled monitoring platforms [4].

This study designs, develops and validates an intelligent sensor-based IoT framework for integrated aquaponics farming as a case study for Sub-Saharan Africa. The proposed system combines multi-parameter sensing, local autonomous control, and web-based monitoring through an ESP32 gateway to support reliable aquaponics operation under resource-constrained conditions.

1.2 Background and Historical Context

- **Global milestones in agricultural technology.**

Smart agriculture has gained market traction in recent years, driven by the integration of IoT, embedded systems, wireless communication and digital monitoring technologies into agriculture [4]. These technologies have changed the way agriculture is conducted, moving it towards a more proactive and data-driven approach. Precision agricultural technologies are spreading, and there is a growing demand for real-time tracking that improves agricultural performance and sustainability through control mechanisms. Because aquaculture and hydroponics are integrated into a closed-loop system, aquaponics has attracted a lot of research interest as an approach to controlled-environment farming. The system produces fish and crops without soil in a limited space and uses less water than conventional soil farming. The most recent studies show a growing trend of integrating IoT into aquaponics systems to enable more precise monitoring, higher levels of automation and better resource management [5], [6].

- **Regional context and development priorities.**

Advanced monitoring and automation have been proven successful in commercially operated aquaponics systems. However, many systems currently in operation are expensive and complex to deploy in developing areas [7]. For example, the cost of a commercially available aquaponics monitoring solution may be too high and out of reach for small-scale farmers in resource-poor areas. There are also special challenges to agricultural technology deployment in Sub-Saharan Africa (SSA). The region has a high share of the world's remaining arable land, but small-scale farming still prevails. Modern irrigation systems are not widely used [8], [9]. The irrigation infrastructure is not reliable, and environmental problems constrain agricultural production in many places.

- **Alignment with international frameworks.**

For international development frameworks, this study aligns with the Sustainable Development Goals set by the United Nations, including SDG 2 (Zero Hunger), SDG 6 (Clean Water and Sanitation) and SDG 7 (Affordable and Clean Energy) [10], as the proposed system strives to boost food production while minimising water consumption and the use of fossil fuels. At the OIC level, the Islamic Organization for Food Security has taken up food-security initiatives focused on OIC countries in Africa [11]. Islamic University of Technology is under the OIC, and some of the OIC member countries are in Sub-Saharan Africa, which makes this research relevant to these current concerns. Although the study

focus is on Sub-Saharan Africa and not on Bangladesh specifically, Bangladesh's own Delta Plan 2100 also highlights the importance of sustainable water resource management and long-term food security planning [12], which are general themes that also drive this study.

- **From traditional farming to intelligent farming systems.**

There is a clear demand for low-cost, durable and sustainable aquaponics solutions powered by the Internet of Things (IoT) that can provide essential monitoring and automation capabilities without the cost of commercial platforms. These conditions give rise to a need for sustainable agricultural technologies that can be used in situations of limited resources. The combination of aquaponics and IoT technologies is a promising approach to addressing these challenges. However, to operate effectively in SSA, it is not enough to have good sensing and monitoring; reliable local operation with intermittent internet and power supply is also required. To this end, the research aims to develop an intelligent sensor-based IoT framework for an integrated aquaponics system in Sub-Saharan Africa, incorporating multi-parameter monitoring, local autonomous control and web-based supervision.

1.3 Problem Statement: Non-Technical Perspective

- **Who is affected.**

This problem mostly impacts farmers engaged in aquaponics in Sub-Saharan Africa. Aquaponics combines fish farming and soilless plant growing in a controlled environment, but to be successful it must be monitored and managed continuously, and several biological and environmental parameters must be controlled. Typical aquaculture parameters measured in conventional aquaponics operations include water temperature, pH, nutrient levels, water quality and environmental conditions, all of which are measured manually [13]. This requires farmers to monitor the system constantly themselves. The manual approach is labour-intensive and may not detect the rapid changes that affect fish health and plant growth.

- **Why this matters now.**

The problem requires immediate attention because agriculture in Sub-Saharan Africa is already challenged by food security issues, limited access to modern agricultural technologies, poor infrastructure, climate change, and a reliance on traditional farming that is constrained by the need for favourable soil conditions, access to water and constant human supervision. Aquaponics can overcome some of these constraints through more

efficient use of water and available space, but its benefits are only realised if the system is monitored properly. Current aquaponics applications have shown that farming efficiency can be enhanced with sensor networks and automation, but most of these systems are working prototypes, and engineering aspects such as communication reliability, response time and energy consumption have not yet been evaluated. Furthermore, many of these solutions rely on cloud connectivity, and this is a limit in Sub-Saharan Africa specifically: only 27 % of Sub-Saharan Africa's overall population is actively using mobile internet [14], while around 790 million individuals are offline despite being under coverage. Internet penetration rates in rural areas are significantly lower than those in urban areas, at 23 % and 57 % respectively, in 2024, according to ITU data. This indicates that a system with continuous cloud connectivity is not feasible for many of the farmers this research aims to serve.

- **Consequences of inaction.**

With manual monitoring, the effects on aquaponics systems are mainly operational and economic. Manual nutrient control, aeration systems and pumps create a risk of inconsistent system performance and delayed nutrient response. This directly affects the health of the fish and plants, which are the two things the farmer depends on for a living and for food. Manual observation may not detect changes in water quality and temperature quickly enough. Relying on a cloud-based solution that does not work when the internet is down poses another operational risk, because the solution can be disabled when it is most needed.

1.3.1 Limitations of Existing Work

Previous IoT-based aquaponics systems have shown that the use of embedded sensors, wireless communication and automated control can improve the monitoring and management of agricultural activities. Some studies have designed systems that measure the most important parameters such as water temperature, pH, nutrient levels and environmental conditions, and control basic farming operations with connected devices [5], [6]. Despite these developments, several limitations remain.

- **Performance is rarely measured.**

Many current aquaponics IoT research papers focus on demonstrating the system and give less attention to communication performance indicators. Some of these systems achieve the monitoring and automation component but do not provide much information about other relevant performance metrics such as latency, packet delivery ratio, system response time and energy consumption. Without quantitative analysis of these parameters, it is

hard to assess the reliability and scalability of the system, and therefore the system's quality. Moreover, certain commercial aquaponics monitoring systems require substantial equipment and resources, making them unaffordable for small-scale farmers in resource-scarce areas.

- **Cloud-only designs.**

The majority of the systems reviewed use a single microcontroller directly connected to a cloud platform and do not use local embedded processing and an additional communication layer. These systems are typically not designed for a hybrid configuration in which the sensing and control are managed by one controller and the wireless communication and web-based monitoring are managed by another. Consequently, they do not benefit from the reliability advantages obtained by splitting the two functions, and none of the systems reviewed include capabilities for over-the-air firmware updates, a REST API, or a locally hosted web dashboard alongside their sensor monitoring features.

- **Fixed thresholds, no adaptation.**

The reviewed systems are primarily based on fixed or basic thresholds, not on adaptive control that can vary with changing conditions. The response time and communication latency of these systems are also not reported, so it is not possible to verify the speed with which they detect and react to a significant change, such as a sudden drop in water quality. Although these systems are described as automated, it is hard to determine whether they can react quickly enough to maintain fish health and plant growth in practice, without this information.

- **Built for a different environment.**

Most current IoT agricultural applications rely heavily on cloud platforms for data management and control. Cloud platforms offer advantages such as remote access and data visualisation, but during periods of unreliable internet connection, a cloud-based architecture can present reliability issues. A system that functions locally and autonomously, while still being monitorable over the network, is better suited to environments that do not always have access to the network or to a reliable power supply. This is the environment in Sub-Saharan Africa.

- **Regional mismatch.**

None of the studies reviewed was developed specifically for Sub-Saharan Africa. They make assumptions about power supply, connectivity and component selection that suit research environments with reliable access to electricity and internet, which are not available

in many parts of Sub-Saharan Africa. This is a regional mismatch while these systems work well in their original test areas, they may not be usable or accessible for farmers in the region targeted by this research.

From the existing literature, these limitations indicate the need for an IoT aquaponics framework that integrates multi-parameter sensing, local autonomous control, web-based monitoring and quantitative performance evaluation. Such a framework can improve the reliability of the system and offers a more viable approach for sustainable deployment of aquaponics systems in difficult environmental and infrastructural settings.

1.3.2 Motivation of the Project

This study is driven mainly by the demand for sustainable and affordable agricultural technologies that can contribute to food production in resource-limited areas such as Sub-Saharan Africa. Aquaponics is a good way to farm fish and grow plants without soil, but it needs ongoing monitoring and management of multiple environmental and water-quality factors.

Current commercial and research aquaponics monitoring systems may be high-cost, complex to set up, or require a constant internet connection, which makes them difficult to use in developing areas. There is therefore a need for an affordable and reliable system that can deliver monitoring and automation capabilities suitable for an environment with an unreliable internet connection and power supply.

This research is driven by the possibility of creating an intelligent IoT-based aquaponics system with embedded autonomous control, multi-parameter sensing and web-based monitoring using commercially available hardware components. The proposed system is designed to improve the reliability and accessibility of automated aquaponics management for small-scale aquaponics applications in Sub-Saharan Africa, while retaining local decision-making.

1.4 Research Gaps: Technical Perspective

The following technical gaps are identified from the study of existing IoT-based aquaponics systems discussed in Section 1.3.1 and the literature reviewed in Chapter 2.

1. **Gap 1: Limited quantitative performance evaluation of IoT-based aquaponics systems.** Current research on aquaponics and its IoT applications has focused mainly on demonstrating monitoring and automation capabilities. Only a small number of studies report detailed engineering metrics such as communication latency, packet delivery ratio, system response time and energy consumption. The absence of quantitative assessment

makes it difficult to judge the reliability and feasibility of such systems, particularly in resource-limited environments where both communication and power are constrained.

2. **Gap 2: Absence of a trustworthy local-control and network-based monitoring system for aquaponics applications.** The vast majority of current IoT agricultural systems rely on the cloud for data management and for the control decisions themselves. While cloud platforms offer remote access, this dependence introduces a risk of system failure wherever internet connectivity is poor. A hybrid architecture in which autonomous control is implemented on-site and network-based monitoring is provided as an additional layer has been investigated only to a limited degree in aquaponics systems, and rarely in the context of deployment in Sub-Saharan Africa.

1.5 Research Questions

The research questions are built directly on the identified research gaps. Gap 1 is answered by RQ2 and Gap 2 is answered by RQ1, so each question maps onto one gap and one objective.

1. **RQ1:** How can a multi-parameter IoT framework provide autonomous local control and web-based monitoring for an integrated aquaponics system under intermittent connectivity and power?
2. **RQ2:** How does the developed IoT-based aquaponics framework perform in measurable aspects such as communication latency, packet delivery ratio, system response time and energy consumption, compared with reported performance metrics from the existing literature?

1.6 Problem Identification: Technical Statement

Aquaponics requires sustained and careful monitoring for the optimal growth of both fish and plants, but small-scale implementations in Sub-Saharan Africa are currently dependent on manual inspection. This is a slow process, and action is often taken too late to prevent losses. Existing IoT solutions offer a potential solution, but they address neither of the two issues identified below.

The first issue is that the reviewed systems report few or no relevant engineering performance metrics, such as latency, packet delivery ratio, response time and power consumption. Without these metrics, it is impossible to evaluate real-world performance.

The second issue is that most existing systems are cloud-controlled and unable to sustain any meaningful activity without an internet connection. Designing a system around a local

controller capable of operating independently of the network would solve this issue, but no such design has been recorded in this specific domain.

This thesis aims to solve both problems by designing and implementing a locally controlled system and by reporting its measured performance against the benchmarks found in the literature.

1.7 Research Aim and Objectives

Aim. The aim of this study is to design, develop and evaluate the performance of a multi-parameter intelligent sensor-based Internet of Things (IoT) framework for an integrated aquaponics farming system, considering the operating conditions and challenges relevant to Sub-Saharan Africa.

- **Objective 1:** To design and develop an intelligent aquaponics system using IoT with multi-parameter sensing, embedded threshold-based hysteresis control and wireless web-based monitoring for automated management of an integrated aquaponics system.
- **Objective 2:** To assess and evaluate the performance of the developed system in terms of communication latency, packet delivery ratio, system response time, energy consumption and operational stability, and to compare the results with relevant benchmarks in the existing literature.

1.8 Expected Outcomes

The study is expected to produce the following outcomes.

1. **Outcome 1:** An intelligent sensor-based IoT aquaponics system for real-time monitoring and control of multiple parameters. The system is built using the Arduino Mega and ESP32, and is operated autonomously within the local area and remotely from the SAOS dashboard on the web.
2. **Outcome 2:** Test and evaluation of the proposed framework in terms of communication latency, packet delivery ratio, system response time, energy consumption and operational stability. The outcomes provide measurable knowledge of the reliability of the IoT-based aquaponics system and its feasibility in resource-constrained areas.

The proposed framework is intended to support sustainable agriculture as an affordable and flexible option for monitoring and automation of small-scale aquaponics applications. It can also serve as a starting point for the adoption of smart farming technologies in regions where manual supervision of resources is still common, such as Sub-Saharan Africa, and contribute to the management of local resources.

1.9 Methodology: Technical Blueprint

This research is based on four main phases: system design, hardware and software development, prototype implementation and performance evaluation.

1. **Phase 1:** This phase identifies the requirements of an IoT-based aquaponics system, including the selection of sensors, actuators, communication protocols and control algorithms. The hybrid embedded architecture is designed using an Arduino Mega 2560 for sensing and local control, and an ESP32 for wireless communication and web-based monitoring.
2. **Phase 2:** This phase covers software and hardware development and integration. The system uses multiple sensors to monitor water quality, environmental conditions and system parameters, and threshold-based hysteresis logic to control the actuators. The ESP32-based SAOS dashboard is developed for real-time monitoring, manual control and access to system data.
3. **Phase 3:** In this phase the prototype is built and tested in the laboratory. Integration is realised through sensor data acquisition, actuator control, communication between the two controllers, and web-based supervision.
4. **Phase 4:** The developed framework is tested against the four measurable parameters of communication latency, packet delivery ratio, system response time and energy consumption, as well as operational stability. The results are examined to assess the credibility and applicability of the proposed IoT framework to a resource-limited aquaponics application.

1.10 Novelty, Contribution, and Importance of the Work

This study is novel in designing and testing an intelligent aquaponics framework using the IoT under a hybrid embedded architecture that combines multiple parameter sensors, local independent control and web-based monitoring. Conventional monitoring systems depended on direct manual observation of the key aquaponics parameters, while the proposed system provides both continuous data acquisition and automatic control of those parameters.

The main contributions of this work are the following.

- **Hybrid embedded IoT architecture:** A dual-controller design is proposed, comprising an Arduino Mega 2560 and an ESP32. The Mega supervises local operations, and the ESP32 facilitates wireless communication and web-based monitoring. A more reliable mode of operation is achieved, which is local in nature but subject to remote supervision.

- **Quantitative performance evaluation:** The proposed framework is experimentally assessed using engineering metrics such as packet delivery ratio, communication latency, system response time, energy consumption and system stability, in order to evaluate the reliability and usability of the deployed system.
- **Application in resource-constrained environments:** The framework is low-cost and flexible, using hardware components that are locally available and can be used in small-scale aquaponics setups in resource-poor areas such as Sub-Saharan Africa.

The impact of this research is its potential to provide a practical solution that improves aquaponics monitoring and automation while addressing reliability, accessibility and system evaluation.

1.11 Significance and Rationale

- **Community Impact:**

This study contributes to the development of an affordable, intelligent aquaponics system for resource-poor areas. The proposed system combines multi-parameter sensing, automatic control and web monitoring, which can support the efficient management of aquaponics farms. The framework can be implemented on small-scale aquaponics systems and provides increased observability, allowing the operator to focus on the critical conditions of the system while reducing reliance on manual monitoring.

- **Scalability:**

The outcomes of this study can be used as a design template for a smart farming solution that is more cost-effective, flexible and easier to use in developing countries, including Sub-Saharan Africa, where infrastructure is limited, cost is high and access to farming technologies can be a challenge.

- **Technical Depth:**

The technical aspect of this study is the application of embedded systems and IoT technology to real-time agricultural monitoring and automation. The hybrid architecture of the Arduino Mega and the ESP32 has worked well for both communication and control, providing reliable local operation while also offering a user interface via the web. The performance evaluation also provides useful information about communication reliability, response time and energy usage of an agricultural IoT system in this context.

- **National and Global Alignment:**

The study is aligned with national and international development priorities. It has a direct positive impact on the United Nations SDGs, particularly SDG 2 (Zero Hunger), SDG 6 (Clean Water and Sanitation) and SDG 7 (Affordable and Clean Energy). The proposed framework enhances food security, reduces water use and promotes the use of solar energy. It is also aligned with the priorities of the OIC and the work of the Islamic Organization for Food Security on improving food security in OIC member countries in Africa. Islamic University of Technology is an OIC institution, and the region under study in this research is Sub-Saharan Africa, where many OIC member states are located.

1.12 Organization of the Thesis

The thesis consists of eight chapters.

- **Chapter 1 — Introduction:** Presents the background, the problem statement, the research gaps, the research questions, the expected outcomes, the methodology blueprint, the objectives, the novelty and the significance of the research.
- **Chapter 2 — Literature Review:** Reviews existing work on aquaponics farming systems, IoT-based monitoring and control, wireless communication for agricultural IoT, and cloud-based data management.
- **Chapter 3 — Methodology:** Describes the materials used in the prototype, the baseline control approach and the four engineering metrics used to evaluate the system.
- **Chapter 4 — Proposed System / Model:** Presents the five-layer system architecture, the detailed design of each layer, the control laws for all ten actuators, and the implementation parameters needed to reproduce the prototype.
- **Chapter 5 — Implementation:** Describes the physical construction of the aquaponics rack and the solar power subsystem, the integration of the Mega and ESP32 firmware with the SAOS dashboard, and the MATLAB/Simulink verification of the control law.
- **Chapter 6 — Results and Discussion:** Reports the measured communication latency, packet delivery ratio, system response time and energy consumption, and compares them against the closest available benchmarks from the literature.
- **Chapter 7 — Conclusion and Future Work:** Summarizes the major findings and contributions, states the limitations of the present work, and provides directions for future research.

- **Chapter 8 — Demonstration of Outcome-Based Education (OBE):** Demonstrates how the project addresses the Course Outcomes, Program Outcomes, Knowledge Profiles, Complex Engineering Problem attributes and Complex Engineering Activity attributes under Outcome-Based Education.

Chapter 2

LITERATURE REVIEW

2.1 Overview

This chapter provides a review of the relevant existing literature in four areas pertinent to the present research: aquaponics farming and its requirements in terms of operation monitoring; IoT-based monitoring and control of aquaponics; wireless communication technologies for agricultural IoT; and cloud-based data management architecture. The chapter discusses existing systems' strengths and, most importantly, their weaknesses, identifying the research gap that the current study aims to fill.

2.2 Related Work

2.2.1 Aquaponics Farming Systems and Operational Monitoring Requirements

Aquaponics is an innovative concept that combines the principles of aquaculture and hydroponics in a closed-loop system to sustain both fish and plant production via the recycling of waste products. While aquaponics is an attractive concept due to its water conservation and nutrient recycling capabilities, it is a highly complicated system that requires continuous and accurate monitoring of its physical and biological parameters. The previous research has shown that the proper monitoring and control of water parameters such as temperature, pH, and nutrient levels are critical for the stable functioning of an aquaponic system [13], [15]. Due to the complexity of interactions in such systems, they require constant attention in order to avoid destabilization of any of their components. Recent studies have shown that monitoring and controlling these parameters using automated systems can be highly beneficial for maximizing the outputs of aquaponic systems [4].

Modern developments in controlled-environment, agriculture have demonstrated the potential of integrating advanced technologies into agricultural settings to optimize production processes. Monitoring technology in agriculture has been recognized as a pivotal element in

transitioning from conventional, observational practices to more systematic, automated forms of crop and livestock management [16].

2.2.2 IoT-Based Monitoring and Control in Aquaponics

The adoption of Internet of Things (IoT) technologies has enhanced monitoring and control capabilities of aquaponics systems through continuous data acquisition, wireless communication and automated decision-making. A number of works have proposed sensor-based IoT frameworks for water quality and environmental monitoring in aquaponics applications.

Wan et al. presented a modularized IoT monitoring system for aquaponics comprising of multiple sensors and edge-computing modules to enhance data collection and monitoring capabilities [5].

In a similar vein, Alselek et al. proposed a water IoT monitoring system for aquaponics and fishery applications, highlighting the potential of embedded sensing systems to monitor system health and environmental conditions [7].

Pramono et al. developed an IoT-based water quality monitoring system using microcontroller-based technology and highlighted the scope of adopting low-cost IoT frameworks for aquaponics monitoring [6].

In addition to aquaponics-specific applications, IoT-based agricultural monitoring systems have been widely explored [4] through the adoption of distributed sensors, wireless communication and automated processing of data. These works highlight the potential of IoT technologies to support agricultural decision-making and resource management. However, existing aquaponics IoT systems primarily focus on monitoring capabilities, while detailed engineering analysis of communication reliability, response performance and energy consumption are limited. Moreover, a number of IoT systems rely heavily on the availability of internet connectivity and cloud-based services for monitoring and control purposes. While such approaches enable remote access to data, this arises reliability concerns in areas with intermittent connectivity. This points to the need for IoT frameworks which can balance local autonomous operation and internet-based monitoring.

2.2.3 Wireless Communication Technologies

Wireless communication technologies play a crucial role in IoT-based agricultural applications by providing communication facilities between sensors, controllers and monitoring platforms. The choice of a proper communication technology relies on the demanded application requirements, such as communication range, data transmission capabilities, energy efficiency, and infrastructure availability.

Wi-Fi is one of the most common adopted technologies employed in IoT applications due to its substantial data transmission capabilities, significant infrastructure availability, and the opportunity to integrate with online monitoring platforms such as the ESP32 [17]. However, it is worth noting that Wi-Fi usually implies increased power consumption when compared with other wireless communication technologies.

ZigBee is a low-power wireless communication technology standardized for low-power sensor networks. ZigBee offers stable and reliable communication channels at relatively short ranges and is often utilized in systems employing multiple sensors with limited energy resources [18]. However, due to its limited range, ZigBee networks typically require additional hardware such as ZigBee routers to establish longer distance communication.

The LoRa and LoRaWAN technologies are specialized in long-range and low-power, making them appropriate for agricultural applications where sensors are distributed across large areas and energy resources are limited. However, LoRa and LoRaWAN technologies offer lower data transmission rates when compared to other communication technologies, which may limit their applications in high-rate data exchange scenarios [19].

The Bluetooth communication standard is primarily focused on short-range wireless communication and configuration scenarios, typically offering limited energy consumption at short-range connectivity. The technology is often integrated into short-range IoT applications and is generally available in most personal computer devices [20].

In the case of small-scale aquaponics, applications with limited monitoring requirements such as Wi-Fi offers an opportunity to employ an easily accessible communication technology, which also integrates with low-cost embedded controllers such as the ESP32. In general, the selection of an appropriate wireless communication technology relies on the specific requirements of the application, which include power consumption, data transmission demands, infrastructure availability, and system characteristics.

2.2.4 Cloud-Based Data Management

Cloud-based platforms for data management are extensively utilized in IoT applications related to agriculture due to their capacity to enable remote storage of data, as well as offer web and mobile access for its visualization. These solutions provide the ability to transmit, store, and analyze data generated by sensors located in agricultural fields through a remote interface. A number of IoT platforms have been deployed for the implementation of monitoring applications in the domain of agriculture. Firebase offers cloud-based database solutions that provide the means for implementing real-time data synchronization between devices and applications. Blynk introduces an IoT platform that enables the development of mobile and web applications for monitoring and controlling devices. ThingsBoard and Ubidots introduce additional solutions for IoT management

[21], [22] that provide extended sets of tools for device management, visualization of data, and development of dashboards. While cloud-based approaches offer significant benefits in terms of enabling remote access to data and processes, solutions that rely exclusively on them may not be suitable for all environments due to the potential limited availability of resources. Connectivity issues can arise in areas that provide limited network coverage. Therefore, IoT applications in the agricultural domain should favor solutions that are based on a combination of local and remote processes that would enable basic functionalities even in the case of the unavailability of the network [23].

2.3 Limitations of Existing Work

While the use of IoT technologies in aquaponics monitoring and automation has been proven in previous research, several challenges remain. There are limited efforts to evaluate in detail the performance of existing systems, which are typically designed as functional prototypes with capabilities for sensor monitoring and basic automation. Many engineering parameters that are critical for the system are not always fully considered, including communication latency, packet delivery ratio, system response time, and energy consumption.

In addition, several agriculture-related IoT systems rely heavily on cloud platforms for data storage, monitoring, and control functions. Cloud services offer remote access, but this can introduce a risk of poor system performance in networked environments with poor-quality internet connections.

The deployment of IoT-based aquaponics systems in resource-constrained areas should therefore consider cost, reliability, and autonomous operation.

2.4 Research Contribution: Addressing and Resolving the Existing Research Gap

In this thesis, limitations of the current aquaponics-based IoT systems are identified, and this study contributes to the development and evaluation of an intelligent IoT framework based on smart sensors for integrated aquaponics farming.

The key contributions are summarized as follows.

1. **Hybrid embedded IoT architecture:** hybrid architecture with an embedded controller (Arduino Mega 2560) with a wireless communication module (ESP32) has been developed to realize local sensing and autonomous control functions while the ESP32 module provides wireless communication and web-based monitoring.

2. **Quantitative performance evaluation:** The developed framework provides measurable performance analysis based on communication latency, packet delivery ratio, system response time, energy consumption, and operational stability, which is not reported in detail in previous aquaponics IoT studies.
3. **Resource-constrained deployment consideration:** The design of the system takes into account the affordability, accessibility, and reliability needs of small-scale aquaponics systems in areas with infrastructure constraints, such as Sub-Saharan Africa.
4. **Practical prototype implementation:** The proposed framework is realized with commercially available hardware components and validated by a real aquaponics prototype, showing the feasibility of an intelligent approach for monitoring and controlling the system for sustainable farming applications.

The characteristics of wireless communication technologies vary with deployment conditions, network configuration, and hardware implementation. The comparison in Table 2.1 is based on reported characteristics of ZigBee, LoRa/LoRaWAN, and Wi-Fi communication standards [17], [18], [19].

Table 2.1: Comparison of wireless communication technologies.

Comparison of wireless communication technologies for IoT-based agricultural monitoring. Characteristics are drawn from the cited specifications and surveys [17], [18], [19].

Parameter	ZigBee	LoRa / LoRaWAN	Wi-Fi
Operating frequency	2.4 GHz (commonly used band)	Sub-GHz bands (433/868/915 MHz, depending on region)	2.4 GHz and 5 GHz bands
Communication range	Short-range, suitable for local sensor networks	Long-range, suitable for distributed monitoring applications	Short-to-medium range, depending on environment and network conditions
Data rate	Low data rate, suitable for sensor data transmission	Low data rate, optimized for energy-efficient long-distance communication	Higher data rate, suitable for internet-connected applications
Power consumption	Low-power operation for sensor networks	Very low-power operation for long-term remote sensing	Higher power requirement compared with low-power protocols
Typical application	Wireless sensor networks and low-power monitoring systems	Large-area agricultural and environmental monitoring	IoT devices requiring direct network connectivity and web access
Suitability for the proposed aquaponics system	Requires additional network infrastructure for deployment	Suitable for large-area applications where long communication distance is required	Suitable due to availability, internet compatibility, and ESP32 support

Chapter 3

METHODOLOGY

3.1 Overview

This chapter explains how the aquaponics monitoring and control system was designed and how its performance was measured. Section 3.2 lists the materials used in the prototype, including the sensors, actuators, controllers and the solar power subsystem, along with the reasons each part was chosen. Section 3.3 describes the baseline control approach and the threshold-based hysteresis control model that replaced it, together with the system state representation. Section 3.4 defines the four metrics used to evaluate the system: communication latency, packet delivery ratio, system response time, and energy consumption. The implementation of this design is described in Chapter 5, and the measured results are reported in Chapter 6.

3.2 Dataset / Materials

The materials used in this work are the physical components of the aquaponics prototype and the sensor channels through which the system is measured. The prototype is based on an Arduino Mega 2560 microcontroller board as the local controller and an ESP32 microcontroller board as the local Wi-Fi gateway and web dashboard host. Seventeen parameters have been measured from the sensor set given in Table 3.1. Ten actuators have been interfaced using relay modules and their functions are given in Table 3.2. A 100 W solar panel, an MPPT charge controller, a 12 V 20 Ah lead-acid battery and a buck converter form the power subsystem. The DS1307 real-time clock is used to timestamp data and to schedule the grow-light and dosing cycles. All components were chosen for low cost, local availability and compatibility with the 5 V and 3.3 V logic levels used by the two controllers, following the reasoning already discussed in the literature on low-cost aquaponics monitoring [7].

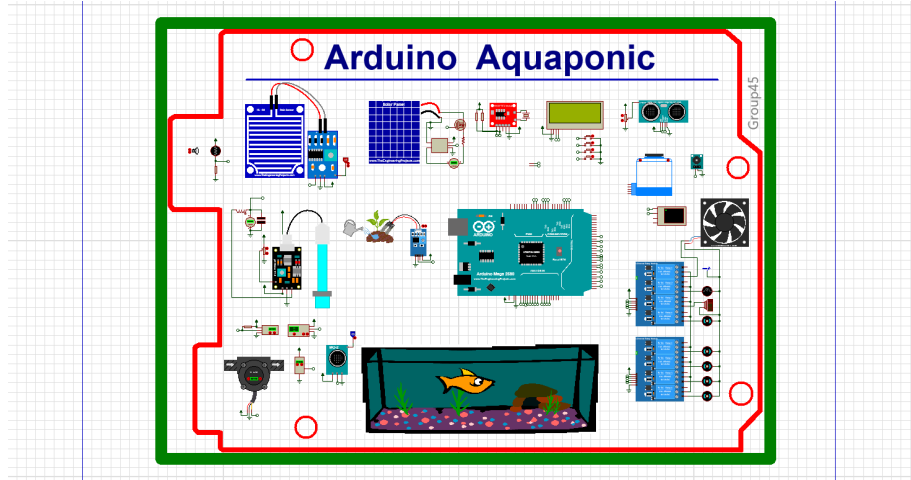


Figure 3.1: Proteus schematic of aquaponics prototype.

Proteus schematic of the prototype showing the Arduino Mega 2560, the ESP32, the relay bank, the sensor connections and the solar power subsystem.

Table 3.1: Measured parameters and their sensor channels (17 parameters).

#	Measured Parameter	Sensor / Source	Interface
1	Water temperature (fish tank)	DS18B20 (probe 1)	1-Wire, D19
2	Water temperature (grow bed / ambient)	DS18B20 (probe 2)	1-Wire, D19
3	Water acidity	pH sensor (analogue)	A0
4	pH temperature compensation	pH temperature probe	A1
5	Total dissolved solids / EC	TDS sensor	A2
6	Ammonia (gas-phase, headspace)	MQ-137	A15 (via mux)
7	Air quality (CO ₂ and related gases)	MQ-135	A15 (via mux)
8	Ambient temperature (plant side)	LM35	A6
9	Ambient temperature (secondary)	NTC thermistor	A3
10	Ambient temperature and humidity	DHT11	D4
11	Grow-bed moisture	Capacitive soil probe	A9 / A8 (DO)
12	Water level in the fish tank	Ultrasonic HC-SR04	D6 / D7
13	Water flow rate	Flow sensor (pulse)	D2
14	Light level (lux proxy)	LDR	A4
15	Infrared proximity	IR sensor	A7
16	Rainfall (ambient)	Rain sensor	A5
17	Power subsystem (solar V/I, battery V/I)	Voltage dividers + ACS-712	A10–A13

The MQ-137 and MQ-135 share analogue input A15 through an analogue multiplexer, which switches the ADC input between the two sensors in turn, so only one gas channel is

sampled at a time. The digital output of the MQ-137 is read on A14 for threshold detection.

Table 3.2: Ten actuators controlled by the Mega 2560 through the relay bank.

#	Actuator	Relay Pin	Function
1	pH-Up dosing pump	D53	Adds alkaline solution when pH falls below threshold
2	pH-Down dosing pump	D51	Adds acidic solution when pH rises above threshold
3	Nutrient dosing pump	D49	Adds nutrient concentrate to the grow beds
4	Tank circulation pump	D47	Moves water between fish tank and grow beds
5	Grow light	D41	Provides artificial light for plant growth
6	Exhaust fan	D39	Removes heat and stale air from the enclosure
7	Heater	D37	Maintains water temperature during cold periods
8	Storage pump	D35	Transfers water between storage tank and grow beds
9	Air pump (aerator)	D33	Oxygenates the fish tank
10	Drain pump	D32	Empties the system for cleaning or reset

3.3 Baseline / Initial Approach

Before the hysteresis controller was implemented, the simplest possible control scheme was considered as a baseline: a single-threshold rule. In this rule, the water pump is switched on the moment the measured soil moisture falls below a fixed value, and switched off the moment it rises above the same value. This is the standard approach used in most entry-level aquaponics and irrigation projects [7], and it is the natural starting point because it needs only one number to configure.

The baseline was considered on the grow-bed moisture channel, but no separate switching-count measurement was taken for it. The single-threshold behaviour is discussed here only to motivate the two-threshold hysteresis law that the firmware actually implements. It would behave acceptably as long as moisture stays far from the threshold, but once moisture settles near the set value, slight sensor noise causes the pump to toggle on and off rapidly. This chattering reduces relay life and stresses the pump. It also produces larger than necessary moisture swings, because the pump always tries to follow a single line.

The proposed design replaces the single threshold with a two-threshold hysteresis rule. A lower threshold θ_{low} switches the pump on, and a higher threshold θ_{high} switches it off. Between the two, the controller keeps its previous state, which produces a dead band and eliminates chattering. The width of the dead band is

$$\Delta\theta = \theta_{\text{high}} - \theta_{\text{low}} \quad (3.1)$$

and for this work the values used are $\theta_{\text{low}} = 30\%$ and $\theta_{\text{high}} = 40\%$, giving $\Delta\theta = 10\%$.

The state of the aquaponics system at any sampling instant is written as the vector

$$X(t) = [T_w(t), \text{pH}(t), \theta(t), A(t), H(t), u(t)] \quad (3.2)$$

where T_w is water temperature, pH is water acidity, θ is grow-bed moisture, A is ammonia concentration, H is ambient humidity, and $u \in \{0, 1\}$ is the pump control input. Every entry in this vector is taken directly from one of the seventeen measured parameters of Section 3.1, so the state model and the sensor set refer to the same quantities. The hysteresis law applied to the moisture term $\theta(t)$ is

$$u(t) = \begin{cases} 1, & \theta(t) < \theta_{\text{low}} \\ 0, & \theta(t) > \theta_{\text{high}} \\ u(t-1), & \theta_{\text{low}} \leq \theta(t) \leq \theta_{\text{high}} \end{cases} \quad (3.3)$$

which is the same rule evaluated in the MATLAB/Simulink model described in Section 5.3.4 and coded into the Mega firmware. The two-threshold form is the smallest change that removes chattering, and it is the control law on which the rest of the system is built.

The same two-threshold pattern is applied to the other controlled quantities of the system. The pH dosing pumps, the nutrient pump, the exhaust fan, the grow light and the heater each operate on their own lower and upper thresholds, so that no actuator is toggled by sensor noise alone. The detailed threshold values and the corresponding control laws are given in Chapter 4, where the proposed system is described in full.

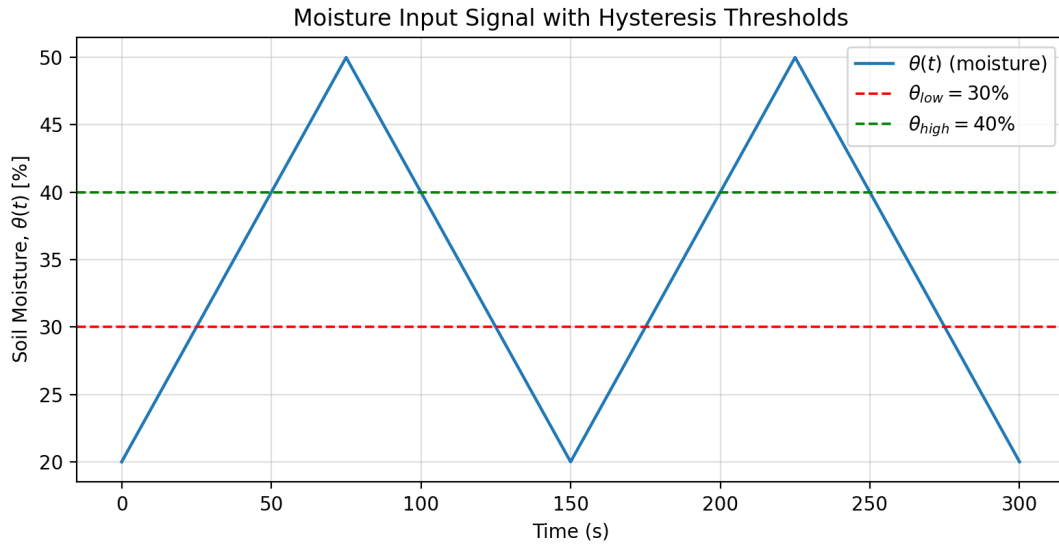


Figure 3.2: moisture thresholds

3.4 Evaluation Metrics

Four engineering metrics are used to evaluate the framework: communication latency, packet delivery ratio, system response time, and energy consumption. Each metric is defined below with its formula and the range of values that indicates acceptable performance. The same definitions are used in Chapter 6, so the quantities reported there can be compared directly with the values discussed here.

3.4.1 Communication Latency

Latency is the time taken for one telemetry packet to travel from the Mega 2560 to the ESP32 and to be written to the SAOS dashboard. The Mega sends one framed packet per second over the UART link, and the ESP32 forwards the parsed fields to the dashboard. Two timestamps are recorded for each packet: t_{send} when the Mega writes the packet to the UART, and t_{receive} when the ESP32 confirms that the corresponding field update has been written to the dashboard. The per-packet latency is

$$L_i = t_{\text{receive},i} - t_{\text{send},i} \quad (3.4)$$

and the mean and standard deviation over N packets are

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N L_i, \quad \sigma_L = \sqrt{\frac{1}{N} \sum_{i=1}^N (L_i - \bar{L})^2}. \quad (3.5)$$

A mean latency of a few milliseconds is expected because the Mega to ESP32 path is a wired UART link. A value above roughly 100 ms would indicate that the ESP32 or the dashboard task is blocked, and a value that grows with N would indicate a buffering problem.

3.4.2 Packet Delivery Ratio

The packet delivery ratio (PDR) is the fraction of telemetry packets that are successfully received at the ESP32 and processed for a dashboard update. A packet is counted as delivered only if it passes the framing check on the ESP32 and its fields are parsed without error; a packet that is malformed, truncated, or not acknowledged within the observation interval is counted as lost. Over a test of N_{total} transmitted packets,

$$R = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\%. \quad (3.6)$$

A PDR above 95% is considered good for a wired UART link, as any loss on this link indicates a framing or buffer problem rather than one related to the radio environment. Lower values may still be useful for monitoring but are flagged in Chapter 6 with the likely culprit.

3.4.3 System Response Time

The response time of the system is the delay between the time a sensor reading exceeds a control threshold and the corresponding actuator changing state. It is computed as

$$T_r = t_{\text{actuation}} - t_{\text{threshold}}, \quad (3.7)$$

where $t_{\text{threshold}}$ is the time when the reading crosses θ_{low} or θ_{high} , and $t_{\text{actuation}}$ is the time at which the relay output changes. Because the control law runs on the Mega, this metric only represents the local decision path; it does not include any network delay. A response time of less than one sampling interval of the controller, i.e., at most 1 s for the current firmware, is considered acceptable.

3.4.4 Energy Consumption

Energy consumption is the electrical energy drawn by the prototype during normal operation. For a component operating at voltage V and drawing current I for a duration t , the energy is

$$E = V \times I \times t, \quad (3.8)$$

and the total for the system is

$$E_{\text{total}} = \sum_{k=1}^n V_k I_k t_k, \quad (3.9)$$

where n is the number of active components. Two values are reported in Chapter 6: the peak load when all actuators are energised, and the average load during normal duty-cycled operation. For the system to run continuously on solar power, the daily energy produced by the panel must meet or exceed the daily consumption,

$$E_{\text{solar}} \geq E_{\text{total}}. \quad (3.10)$$

A design that satisfies this inequality over a full day, including overnight operation, is treated as self-sufficient.

For Sub-Saharan Africa, the daily solar yield is estimated using an average of five peak sun hours per day, a figure commonly used in solar sizing studies for the region [24]. The daily energy produced by the panel is therefore the panel rating in watts multiplied by five hours. For

the 100 W panel used in this work, this gives an estimated daily yield of about 500 Wh. This peak-sun-hour figure is used in Sections 4.3.5, 5.2 and 6.2 to size the solar subsystem against the prototype’s daily consumption.

3.4.5 Test Plan

The four metrics are measured in a single session on the bench with the completed prototype. The latency is recorded over $N = 2000$ consecutive telemetry packets at the nominal rate of one packet every second, which yields a test window of around 33 minutes. A separate 30-minute window was used for the PDR test. During this window the effective transmit rate was approximately 0.55 frames per second, giving a total of $N_{\text{total}} = 1000$ transmitted frames. The lower effective rate reflects the fact that the Mega transmits at one packet per second only when its telemetry scheduler is due, and the counted test window was shorter than the nominal duration. The response time is measured over five trials in which the grow-bed moisture is manipulated across the lower threshold by hand and the relay transition timed. The component power values used in this work are the manufacturer-rated voltages and currents for each module, not bench-measured currents. Two load figures are reported: the nominal operating load, with the components that are energised during normal duty, and the peak load, with all actuators energised at once. These are reported in Appendix B, Table B.4.

Chapter 4

PROPOSED SYSTEM / MODEL

4.1 Motivation

Chapter 2 identified two limitations in the current aquaponics IoT literature. The first is that most reported systems depend on a cloud link for their control decisions, so a lost connection can interrupt water circulation, aeration or dosing [7]. The second is that the same studies rarely report measured engineering performance such as latency, packet loss and energy use, which makes it difficult to judge whether a design will survive in a place with intermittent power and network coverage [5]. The proposed system is built to close both gaps. All control decisions are made on a local Arduino Mega 2560, so the aquaponics loop keeps working even with no Wi-Fi. An ESP32 sits beside it only as a gateway and dashboard host, so the user can still see the system and override it when a connection is available. Chapter 6 then reports the same engineering metrics that earlier studies left out, so the design can be compared on equal terms.

4.2 System / Architecture Overview

The proposed architecture is a five-layer design in which the sensing, control, communication and monitoring functions are kept strictly separate. This separation is what allows the system to keep operating when the network is unavailable: the layers below the network boundary are self-sufficient, and the layers above it are only needed when a user wants to look at the system or override it. The five layers are listed below and shown in Figure

1. **Sensing and actuation layer.** Seventeen measured parameters are acquired by the sensor set described in Section 3.2, and ten relay-driven actuators are wired to the Mega 2560 through the relay bank listed in Table 3.2.
2. **Local control layer.** The Arduino Mega 2560 reads the sensors, runs the threshold-based hysteresis control described in Section 3.3 for every actuator, updates the local LCD and, at a fixed rate of one packet per second, prepares a framed telemetry packet for the gateway. All control decisions are made here, so this layer never depends on the network.

3. **Local link layer.** The Mega and the ESP32 are connected by a wired UART link at 115200 baud. Framed packets in the form $\langle . . . \rangle$ carry telemetry from the Mega to the ESP32, and short control frames carry user commands and manual overrides back from the ESP32 to the Mega.
4. **Gateway and hosting layer.** The ESP32 receives and validates each telemetry packet, parses the fields into its internal data structure, updates its history buffer, and serves the SAOS dashboard together with a small REST API over the local Wi-Fi network. No cloud service is required for this layer to function; it works even when the site has no internet connection.
5. **Optional remote layer.** MQTT and Firebase are provided as optional hooks so that a future deployment can forward the same data to a remote server if internet access becomes available. This layer is not used by the current prototype and is not needed for any of the metrics reported in Chapter 6.

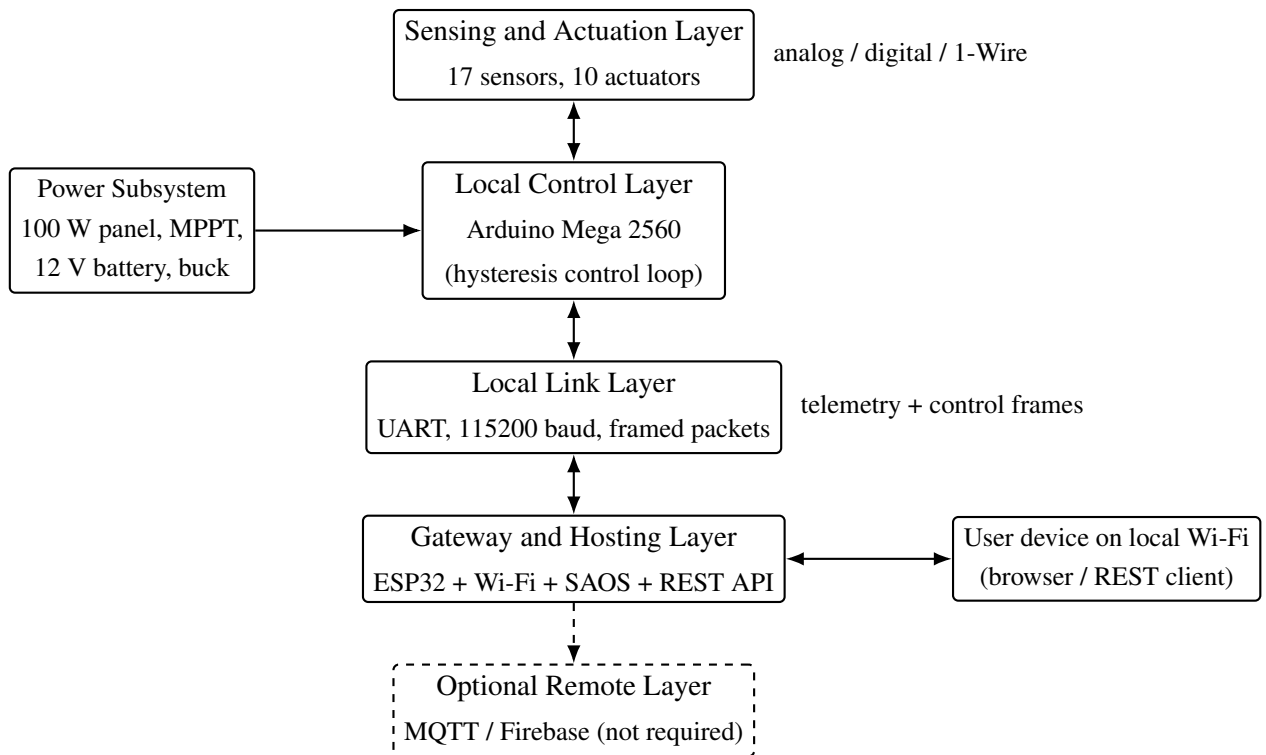


Figure 4.1: Five-layer system architecture.

The proposed system, has sensing and actuation layer which reads the seventeen measured parameters and drives the ten actuators, of which eight are threshold driven. The local control layer (Mega 2560) owns every control decision.

The separation between the local control layer and the gateway layer is the main difference between this design and the cloud-centric systems reviewed in Chapter 2 [6], [7]. It is also the reason why the response-time and latency results in Chapter 6 can be interpreted without worrying about network variability in the control path.

4.3 Detailed Design

This section expands each layer of the architecture into its design details. The subsections follow the five-layer stack of Figure 4.1 so that every element of the diagram is accounted for. Control laws for all ten actuators are described here, since Section 3.3 only covers the pump hysteresis as the representative baseline case. Eight of the ten are threshold-driven; the air pump runs continuously and the grow light follows the real-time clock schedule.

4.3.1 Sensing and Actuation Layer

The seventeen measured parameters of Table 3.1 are read by the Mega 2560 through three interface types: analog inputs for the pH, TDS, LM35, NTC, LDR, rain, soil, IR, gas and power-divider channels; digital inputs for the flow sensor, the ultrasonic module and the digital outputs of the soil and gas modules; and a shared 1-Wire bus for the two DS18B20 probes. The DS1307 real-time clock and the 16×2 I2C LCD are also on this layer, since they read or write directly to the Mega and are needed for timestamping and local display.

The ten actuators of Table 3.2 are driven by the Mega through an eleven-channel relay bank. Each relay is a standard optically-isolated 5 V module, chosen for low cost and for isolating the inductive load of the pumps and the fan from the microcontroller. The buzzer shares the same relay bank but is treated as an alarm output rather than an actuator.

4.3.2 Local Control Layer

The Mega 2560 runs a single non-blocking loop. On every pass it polls the sensors through the sensor manager, updates the display if a page change is due, services the UART for user commands, and evaluates the control laws for all actuators. Threshold values and hysteresis bands were chosen from the acceptable ranges discussed in Chapter 2. The control laws are as follows.

Circulation, storage, and drain pumps. The tank circulation pump uses the two-threshold hysteresis law defined in Equation (3.3), with $\theta_{\text{low}} = 30\%$ and $\theta_{\text{high}} = 40\%$ on the grow-bed moisture channel. The storage pump and the drain pump follow the same law on their own moisture and water-level thresholds, so no pump is toggled by sensor noise alone.

pH dosing. Two dosing pumps correct pH in either direction. The pH reading is first temperature-compensated by the probe on channel A1. If the compensated pH falls below 6.8, the pH-Up pump is switched on; if it rises above 7.4, the pH-Down pump is switched on. Only one of the two pumps can be energised at a time, and each pump runs for a fixed short dose followed by a settling interval, so a single noisy reading cannot move the pH far in one step.

Nutrient (TDS) dosing. The nutrient pump follows the same two-threshold pattern on the TDS channel. It is switched on when TDS falls below the lower set point and switched off when TDS rises above the upper set point. This keeps the nutrient concentration inside the same band that the plants were calibrated for.

Exhaust fan and heater. The exhaust fan is switched on above an upper ambient-temperature threshold and switched off below a lower threshold. The heater uses the mirrored logic on the water-temperature channel. Both follow the two-threshold pattern, so the fan and the heater never toggle near their set points.

Grow light. The grow light is scheduled by the DS1307 RTC rather than by a sensor threshold, since plant growth depends on a photoperiod rather than on a single reading. The light is switched on for the configured daytime window and switched off outside it. The LDR channel is used only to verify that the light is actually on during the scheduled window.

Air pump and buzzer. The air pump runs continuously whenever the system is powered on, since aeration must not depend on any threshold. The buzzer is driven by the control loop only when a reading leaves its safe range, and it is silenced as soon as the reading returns inside the band.

In addition to these automatic laws, the same layer accepts manual commands for any of the ten actuators when the ESP32 forwards them over the UART link, although in normal operation the air pump and the grow light run under their own continuous and scheduled logic rather than through manual override.

4.3.3 Local Link Layer

The Mega and the ESP32 are connected by UART at 115200 baud. The Mega transmits one framed telemetry packet per second, in the form `<field0, field1, . . . , field43>`. The frame format is fixed, so the ESP32 can parse fields by position instead of by name, which keeps the parsing cost low on both sides. The Mega also accepts short framed control packets from the ESP32 for manual commands and mode changes, using the same delimiter convention [23].

4.3.4 Gateway and Hosting Layer

The ESP32 runs a small firmware that (i) receives and validates each UART frame, (ii) parses the fields into an internal data structure, (iii) updates a rolling history buffer for the dashboard charts, and (iv) serves the Smart Aquaponics Operating System (SAOS) dashboard and a small REST API over the local Wi-Fi network. The REST API exposes endpoints such as `/json`, `/api`, `/health`, `/info` and `/developer`, which are intended for development and integration rather than for normal operation. Over-the-air firmware updates are supported on the ESP32 so the device can be updated without a physical connection once the prototype is mounted on the rack.

Because the dashboard and the REST API run on the ESP32 itself, all monitoring and manual override functions remain available to any device on the same Wi-Fi network even when the site has no internet connection. This directly addresses the reliability limitation identified in Chapter 2, where cloud dependency could break the user interface during a network outage.

4.3.5 Power Subsystem

The prototype is powered by a 100 W solar panel, an MPPT charge controller, a 12 V/20 Ah lead-acid battery and a buck converter. The panel is sized so that the daily energy yield at the typical Sub-Saharan peak-sun-hour figure used in Section 3.4.4 exceeds the daily consumption of the prototype. The battery buffers the system overnight and during cloudy periods. The pumps are 3 V DC modules driven through the relay bank from the 12 V bus via on-board regulators; the buck converter provides the 5 V rail for the Mega, sensors, and relay coils, and the 3.3 V rail for the ESP32.

4.4 Implementation Details

The implementation parameters below are the values needed to reproduce the prototype. Full firmware listings are given in Appendix A, and the hysteresis control pseudocode is given in Appendix C.

Hardware. Arduino Mega 2560 (ATmega2560, 5 V logic) and ESP32 (dual-core, 3.3 V logic, Wi-Fi 802.11 b/g/n). Both boards are mounted on the prototype rack described in Chapter 5. The relay bank, sensors and power electronics are the components listed in Tables 3.1 and 3.2.

Communication. Wired UART at 115200 baud between the Mega and the ESP32, with a fixed frame rate of one telemetry packet per second. The framing convention and field order are given in Appendix A.

Firmware versions. Mega firmware SAOS Mega V5.0.7, ESP32 firmware SAOS V1.0.0. Both were built with the Arduino IDE and the ESP32 board package. Firmware sources are provided in the appendix so the two versions can be reflashed on the same hardware set.

Libraries. The Mega firmware uses DHT, RTCLib, LiquidCrystal_I2C, OneWire and DallasTemperature. The ESP32 firmware uses the standard WiFi, WebServer and ArduinoOTA libraries. The complete list with versions is given in Appendix A.

Control parameters. Hysteresis band on the grow-bed moisture channel: $\theta_{\text{low}} = 30\%$, $\theta_{\text{high}} = 40\%$, $\Delta\theta = 10\%$. pH dosing limits: 6.8 to 7.4. Sampling interval of the control loop: 1 s. The same parameters are used in the MATLAB/Simulink verification model described in Chapter 5, so the simulation and the firmware run the same control law.

Simulation. MATLAB/Simulink was used to verify the hysteresis logic before it was coded into the Mega firmware. The model is a three-block structure: an input signal generator, a hysteresis control block with a unit delay for state memory, and a scope for the pump control signal. The model and its parameters are described in Chapter 5, and its output is reported there with the measured results.

Design tools. The circuit schematic was drawn in Proteus, and the Simulink model was built in MATLAB. The prototype photograph and the SAOS dashboard screenshots are given in Chapter 5 as evidence of the assembled system.

Chapter 5

IMPLEMENTATION

5.1 Overview

This chapter describes how the design in Chapter 4 was built and brought up. Section 5.2 covers the physical construction of the prototype such as: the rack, plumbing, sensor placement and solar power subsystem. Section 5.3 also describes how the work was integrated: the (Mega and ESP32) firmware , explains how the two exchanges data over the wired UART link, the SAOS dashboard runs on the ESP32, and the MATLAB/Simulink verification carried out before the control law was coded into the Mega. Section 5.4 states the experimental setup used to obtain the measurements reported in Chapter 6, and that the executed test plan is referred to the same sample sizes and duration's. The measured results themselves are not given here; they are reported in Chapter 6.

5.2 Design / Approach

The prototype is a vertical aquaponics rack with a single fish tank at the base and two grow beds above it, as shown in Figure 5.1. The frame is built from white PVC tubing and MDF shelves. The lowest shelf holds the fish tank, with tilapia fish. The two shelves above it hold the grow beds, with mint and lettuce planted. A small dosing station with three bottles (nutrient, pH-Up and pH-Down) is mounted on the side of the rack and feeds into the main circulation line through the three dosing pumps.

Water is moved between the fish tank and the grow beds by the tank and storage pumps. The air pump runs continuously to keep the fish tank oxygenated. Two small cooling fans are mounted at the top of the electronics compartment to protect the relay bank and the power electronics from heat produced inside the enclosure.

All sensor probes are placed where their reading is representative of the process. The two DS18B20 probes sit one in the fish tank and one in the grow-bed reservoir, since the two temperatures can differ. The pH probe and its temperature-compensation probe sit together in the fish tank at a depth that stays covered during pump cycling. The TDS probe is in the same tank.

The soil-moisture probe is inserted into the upper grow bed. The ultrasonic module is mounted above the fish tank to read the water level. The LDR, rain, IR, LM35 and NTC sensors are on the outside of the rack, so that they measure the ambient conditions around the plants rather than the conditions inside the electronics compartment.

The solar power subsystem is mounted on the same rack. The 100 W panel is on the top shelf, and the MPPT charge controller, the 12 V/20 Ah battery and the buck converter are in the electronics compartment underneath. The panel feeds the MPPT controller, the controller charges the battery, and the battery output is converted to the 5 V and 3.3 V rails that supply the Mega, the ESP32 and the sensors. The pumps and the fans are switched through the relay bank from the 12 V bus. Their rated operating voltage is lower than the bus voltage the pumps are rated at 3 V and the fan at 5 V and the voltage is stepped down at the point of connection to each actuator. The switching is done at the 12 V bus, so the relay contacts carry the full bus voltage even though each actuator itself runs at its rated voltage.

The prototype has two rated load figures, and both are stated on a manufacturer-rated basis rather than as bench measurements. The **nominal operating load**, with the controller, sensors, relay bank, air pump, and grow lights energised and the other actuators idle, is approximately 17.2 W. The **peak load**, with all eight controllable actuators energised simultaneously, is approximately 26.9 W. These states are shown in Table 5.1. Battery autonomy is computed against the nominal load of 17.2 W. Assuming 50% depth of discharge for the lead-acid battery, the usable capacity is about 120 Wh, which gives approximately 7.0 h of autonomous operation at the nominal load. This is sufficient for a shorter night or for a night with reduced load, but it is not sufficient to carry the full nominal load through a full 11–13 h night in the target region without some solar input. Larger usable battery capacity or a reduced overnight load would be required to guarantee uninterrupted operation through a full night. The daily consumption at the nominal load over 24 hours is about 413 Wh, which is comfortably within the daily yield of the 100 W panel at the peak-sun-hour figure stated in Section 3.4.4.

Table 5.1: Rated power for each prototype component.

All currents are manufacturer ratings, not bench measurements. The nominal operating load includes the controller, sensors, relay bank, air pump, and grow lights, with the other actuators idle. The peak load includes every controllable actuator energised at once.

Component	Voltage (V)	Current (A)	Power (W)
Arduino Mega 2560 (active)	5.0	0.15	0.75
ESP32 (Wi-Fi active)	3.3	0.24	0.79
pH sensor module	5.0	0.02	0.10
Gas and ultrasonic sensors	5.0	0.15	0.75
LCD, DHT11, RTC (combined)	5.0	0.03	0.15
Relay bank (11 channels)	5.0	0.14	0.70
Water-quality sensors (combined)	5.0	0.10	0.50
Air pump (aerator, continuous)	3.0	0.50	1.50
LED grow lights (3 tiers)	12.0	1.00	12.00
Nominal operating load	—	—	≈ 17.2
Circulation pump	3.0	0.24	0.72
Storage pump	3.0	0.24	0.72
Drain pump	3.0	0.24	0.72
Nutrient dosing pump	3.0	0.24	0.72
pH-Up dosing pump	3.0	0.24	0.72
pH-Down dosing pump	3.0	0.24	0.72
Exhaust fan	5.0	0.10	0.50
Heater	12.0	0.40	4.80
Peak load (all actuators on)	—	—	≈ 26.9

The complete wiring was brought up in stages. First the Mega was powered on its own and the sensor channels were verified one at a time through the serial monitor. Next the relay bank was connected and every relay was toggled by hand to confirm the assignment between relay channel and actuator. The ESP32 was then connected over the UART link and the framed telemetry was checked on the serial monitor of each board. Finally the SAOS dashboard was brought up on a laptop joined to the same Wi-Fi network and the live readings were confirmed against the serial monitor.

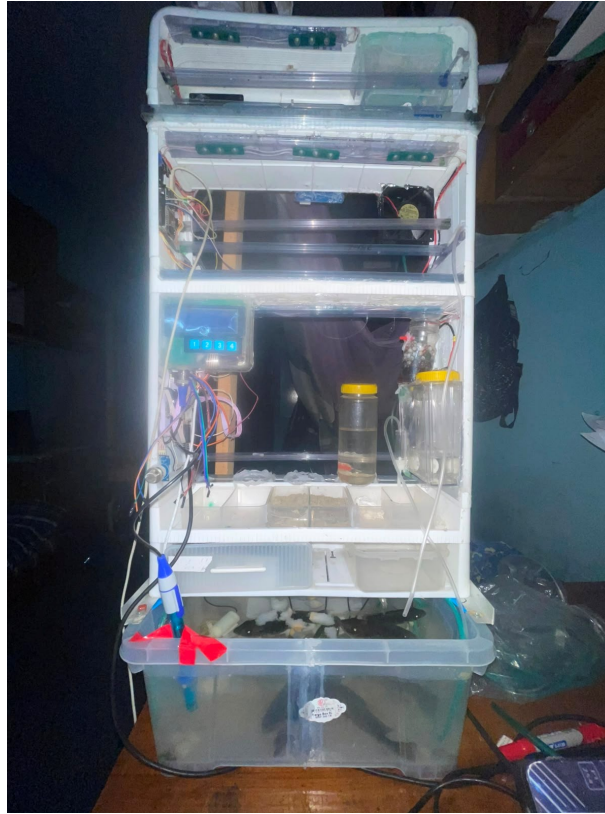


Figure 5.1: Photograph of the vertical aquaponics prototype.

5.3 Integration

Two firmware images are used and they are coupled by a single wired UART link. The Mega firmware runs the local control loop and produces the telemetry; the ESP32 firmware consumes the telemetry, hosts the SAOS dashboard and the REST API, and can send short control frames back to the Mega. The full listings are given in Appendix A, and the hysteresis logic is given as pseudocode in Appendix C.

5.3.1 Mega Firmware

The Mega firmware (SAOS Mega V5.0.7) is organised as a cooperative, non-blocking loop. After a safe start-up sequence that configures the relays and forces every output to a known OFF state, the loop calls four managers in turn on every pass: the sensor manager, the button manager, the display manager and the actuator manager. Sensors are read on their own scheduled intervals, the LCD page is updated only when it changes, and the actuators are evaluated on every pass so that a threshold crossing is acted on as soon as it is seen.

The actuator manager holds the control law for each of the eight controllable actuators

described in Section 4.3. Every law is a two-threshold hysteresis rule with a unit-delay element that keeps the previous state, which is the same structure that was verified in Simulink later in this section. Telemetry is assembled once per second into a framed packet with forty-four numeric fields, using only stack buffers so no dynamic memory is allocated during normal operation. The packet is written to the UART in the same pass, and a duplicate is printed to the serial monitor for bench debugging.

Manual override is supported through the same UART link. When a framed control packet arrives from the ESP32, the Mega parses the command and either switches a single actuator or changes the mode between AUTO and MANUAL. A watchdog returns the system to AUTO if no valid command is received for eight seconds, so a lost gateway cannot leave the system in an unintended manual state.

5.3.2 ESP32 Firmware

The ESP32 firmware (SAOS V1.0.0) has three responsibilities. First, it receives each UART frame from the Mega, validates the framing, and parses the fields into an internal data structure. Invalid frames are counted and dropped, so a corrupted packet cannot poison the dashboard state. Second, it maintains a rolling history of the parsed fields and serves the SAOS dashboard and the REST API to any device on the same Wi-Fi network. Third, it accepts manual commands from the dashboard and forwards them to the Mega as framed control packets.

The ESP32 also runs over-the-air update support so the firmware can be replaced without a physical connection once the prototype is in place. The ESP32 can either join an existing Wi-Fi network or serve its own access point, and in both modes the dashboard is reachable from any device on the same local network without external internet access. The optional MQTT and Firebase hooks described in Section 4.2 are not used by the current prototype and do not affect normal operation.

5.3.3 SAOS Dashboard

The SAOS dashboard is served by the ESP32 and can be opened in any browser on the same Wi-Fi network. It is organised into six pages — Weather, Hydro, Aqua, Power, Charts and System — and each page groups the fields that belong together. Live values update once per second from the UART stream. A diagnostics page allows each sensor channel to be tested individually, and the same page reports the status of the UART link, the Wi-Fi connection and the optional remote hooks. The manual override page lets the user switch any of the eight controllable actuators on or off. The dashboard is the primary user interface of the system and, in combination with the REST API, provides the network-based monitoring that Chapter 2 identified as missing from

most cloud-only aquaponics designs.

5.3.4 MATLAB/Simulink Logic Verification

Before the hysteresis control law was coded into the Mega firmware, it was verified in MATLAB/Simulink. The model has three blocks, as shown in Figure 5.2: an input signal generator, a hysteresis control block with a unit delay for state memory, and a scope for the pump control signal. The input is a repeating sequence that represents a slow variation of grow-bed moisture across the band of interest. The control block implements the two-threshold law of Equation (3.3) with the same thresholds used in the firmware: $\theta_{\text{low}} = 30\%$ and $\theta_{\text{high}} = 40\%$.

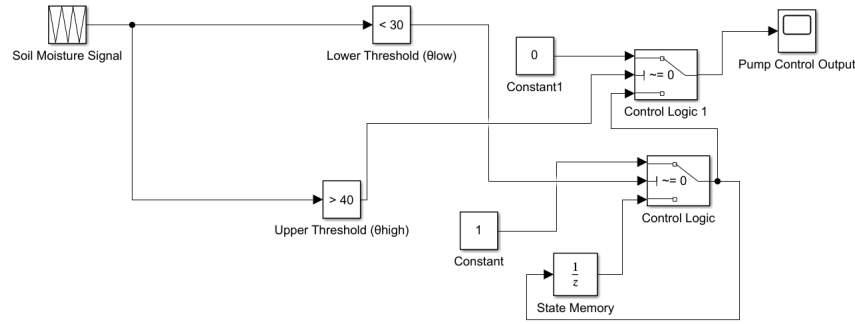


Figure 5.2: MATLAB/Simulink model of the hysteresis control law.

The simulation runs for 300 s with a discrete step of 1 s, which matches the sampling interval of the firmware. The pump control signal $u(t)$ over the run is shown in Figure 5.3. The output is a clean square wave with no chattering, and the transitions happen only when the input crosses the outer thresholds. The dead band between 30% and 40% is respected throughout: the output holds its previous state in every sample that falls inside the band.

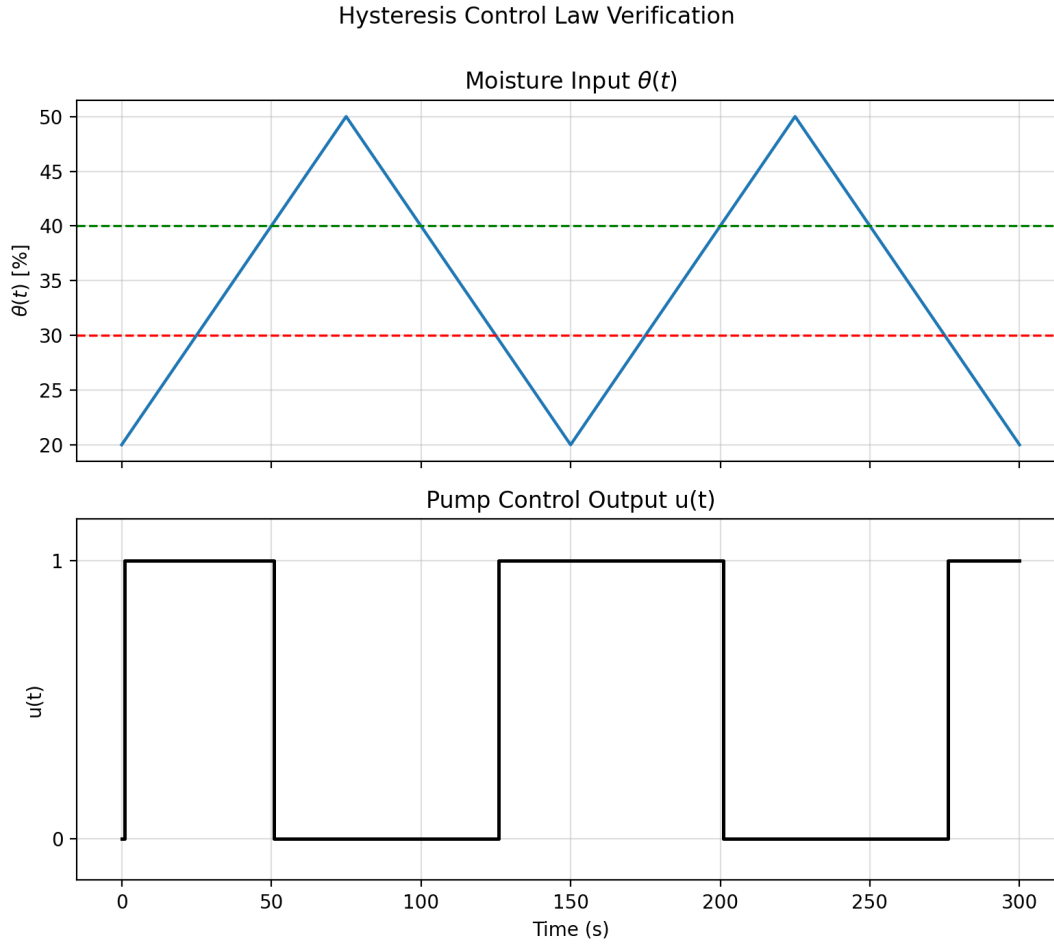


Figure 5.3: Pump control signal output.

Pump control signal over the 300 s simulation run. The top panel shows the moisture input $\theta(t)$ with the lower threshold $\theta_{\text{low}} = 30\%$ and the upper threshold $\theta_{\text{high}} = 40\%$ marked as dashed lines. The bottom panel shows the pump control output $u(t)$ on the same time axis. The output toggles ON when $\theta(t)$ falls below 30% and OFF when it rises above 40%, and it holds its previous state inside the dead band. This is the behaviour the firmware implements.

This verification is a logic check, not a validation of the physical system. It confirms that the control law behaves as designed under noise-free input. Behaviour under real sensor noise, real power conditions and real communication is measured on the physical prototype and reported in Chapter 6. The simulation model and its results are kept separate from the measured results for that reason.

5.4 Experimental Setup

The measurements reported in Chapter 6 were taken in a single bench session on the completed prototype, with the sensors and actuators running from the solar power subsystem and the ESP32 serving the SAOS dashboard over the local Wi-Fi network. The sample sizes and durations below are the same as those stated in Section 3.4, so the plan and the executed tests are identical.

Communication latency. $N = 2000$ consecutive telemetry packets were recorded at the nominal rate of one packet per second, giving a test window of about 33 minutes. For each packet, t_{send} was taken when the Mega wrote the frame to the UART and t_{receive} was taken when the ESP32 confirmed the corresponding field update to the dashboard. Mean latency and standard deviation were computed from these two timestamps over the full run.

Packet delivery ratio. A separate 30-minute window was used for the PDR test. During this window the effective transmit rate was approximately 0.55 frames per second, giving a total of $N_{\text{total}} = 1000$ transmitted frames. A frame was counted as delivered only if the framing check on the ESP32 passed and its fields were parsed without error; malformed frames and frames that were not accepted within the observation interval were counted as lost.

System response time. Five trials were carried out. In each trial, the grow-bed moisture reading was brought across the lower threshold by hand and the time between the threshold crossing and the change of state of the pump relay was recorded. The five values were averaged to give the mean response time.

Energy consumption. The component currents and voltages in Table 5.1 are the manufacturer-rated values for each module, not bench-measured currents. Two load figures are reported. The nominal operating load, with the controller, sensors, relay bank, air pump, and grow lights energised and the other actuators idle, is approximately 17.2 W. The peak load, with all eight controllable actuators energised simultaneously, is approximately 26.9 W. The nominal figure is used for the battery-autonomy calculation and for the daily energy-budget check against the 100 W solar panel, because the peak state is transient and does not occur during normal duty cycling.

Environment. All measurements were taken indoors, at room temperature, with the ESP32 dashboard open on a laptop on the same Wi-Fi network. No long-term field trial was carried out in this work; Section 6.3 discusses what a longer deployment would add and what has not yet been measured.

Chapter 6

RESULTS AND DISCUSSION

6.1 Results and Evaluation

This section presents the measured performance of the prototype described in Chapter 5. Four engineering metrics were recorded during a single bench session: communication latency, packet delivery ratio, system response time, and energy consumption. The conditions under which the measurements were taken are stated in Section 5.4, and the full underlying data are given in Appendix B.

Communication latency. Two thousand telemetry packets were recorded between the Mega and the ESP32. The mean latency was 2 ms with a standard deviation of 1.25 ms, a minimum of 1.5 ms and a maximum of 23.3 ms.

Packet delivery ratio. Of the 1000 packets transmitted during the 30-minute PDR window, 930 were accepted for dashboard update, giving a PDR of 93 %. The 70 lost packets failed the framing check on the ESP32 or were not accepted within the observation interval.

System response time. Five threshold-crossing trials gave a mean response time of 0.029 s, with individual values between 0.019 s and 0.045 s. This is the local decision path only: the Mega reads the moisture channel, evaluates the hysteresis law, and switches the relay without involving the ESP32 or the network.

Energy consumption. The prototype has two rated load figures. The nominal operating load, with the controller, sensors, relay bank, air pump, and grow lights energised and the other actuators idle, is approximately 17.2 W. The peak load, with all eight controllable actuators energised simultaneously, is approximately 26.9 W. Both figures are stated on a manufacturer-rated basis rather than as bench measurements, and the component-level breakdown is in Table 5.1 of Chapter 5.

Table 6.1 brings the four measured metrics together and places them beside the closest available benchmarks from the literature reviewed in Chapter 2. Table 6.2 then compares the proposed system as a whole against the three related published systems.

Table 6.1: Measured results against literature benchmarks.

Metric	Measured (this work)	Benchmark	Source
Communication latency	2 ms (mean)	< 1.5 s	Fathi [22]
Packet delivery ratio	93 %	99.2 % at 90 m	Wan et al. [5]
System response time	0.029 s (mean)	Not reported	—
Energy consumption	17.2 W nominal, 26.9 W peak	Not reported	—

Table 6.2: Comparison of the proposed system against related IoT aquaponics systems.

Feature	Wan et al. [5]	Alselek et al. [7]	Pramono et al. [6]	Proposed system
Sensor parameters	16 channels	Water quality	3 parameters	17 measured parameters
Actuators	Not reported	Not reported	Not reported	10 actuators, individually controlled
Control type	Monitoring only	Monitoring only	Threshold alerts	Hysteresis control with manual override
Controller hardware	Not reported	Not reported	ESP-series	Arduino Mega + ESP32 hybrid
Communication	Wi-Fi	Wi-Fi	Wi-Fi	Wi-Fi (ESP32) + UART (Mega-ESP32)
Local dashboard	No	No	No	Yes (SAOS on ESP32)
Cloud platform	Custom	Custom	Not reported	Optional MQTT/Firebase
OTA updates	Not reported	Not reported	Not reported	Yes
Manual override	Not reported	Not reported	Not reported	Yes
Power source	Not reported	Not reported	Not reported	Solar + battery
Live fish and plants	Not reported	Not reported	Not reported	Yes (fully operational)
SSA-specific context	No	No	No	Yes
Cost level	Not reported	GBP 3,800+	Low	Low (BDT 15,000 budget)

6.2 Discussion

The results reported in Section 6.1 are interpreted here against the two research questions stated in Section 1.5.

Communication latency. The mean latency of 2 ms is consistent with the design expectation for a wired UART link at 115200 baud with a fixed telemetry rate of one packet per second. The small standard deviation of 1.25 ms shows the link was stable across the run. The maximum of 23.3 ms occurred only in a few samples and is consistent with occasional scheduler contention on the ESP32 when the dashboard refresh overlapped with the UART read. The value is far below the benchmark of < 1.5 s reported by Fathi [22], but the comparison is not like-for-like: Fathi measures an ESP8266 to Firebase link over the internet, whereas this work measures a local UART link. The correct reading is that the local telemetry path is fast enough that it never becomes the bottleneck in the control loop, which was the design goal.

Packet delivery ratio. The 93 % figure is a reasonable result for a bench prototype, but it is lower than the 99.2 % reported by Wan et al. [5] over a wireless link at 90 m. Two effects explain the gap. First, the ESP32 dashboard task occasionally blocked the UART read long enough for a frame to be missed, which is a firmware scheduling issue rather than a link issue. Second, the test was run indoors in a shared Wi-Fi environment, so the ESP32 was also servicing unrelated Wi-Fi traffic while reading the UART. Neither effect is fundamental to the architecture, and both can be reduced by moving the UART read to a dedicated task.

System response time. The mean response time of 0.029 s is bounded by the 1 s sampling interval of the Mega control loop and by the relay switching time. It does not include any network delay because the control decision is made entirely on the Mega. No comparable response-time measurement was found in the three closely related aquaponics studies reviewed in Chapter 2, so this metric is one of the direct contributions of the present work.

Energy consumption. The nominal load of 17.2 W and the peak load of 26.9 W are both stated on a manufacturer-rated basis. They describe the expected load of the prototype rather than a bench-measured current, and the difference between them reflects the fact that the peak state is transient and does not occur during normal duty cycling. The nominal figure is the one used for the battery-autonomy calculation and for the daily energy budget. The 100 W panel produces roughly 500 Wh per day at the five peak sun hours stated in Section 3.4.4, and the daily consumption at the nominal load over 24 hours is about 413 Wh, which is comfortably within the panel's daily yield. Battery autonomy at the nominal load is approximately 7.0 h at 50% depth of discharge, which is not sufficient to cover a full 11–13 h night at the nominal load, but the overnight draw is in practice much lower because the grow lights are off and most actuators are idle. A larger battery or a reduced overnight duty cycle is identified as future work in Section 7.3.

Answering the research questions. RQ1 asked how a multi-parameter IoT framework can provide autonomous local control and web-based monitoring under intermittent connectivity and power. The local control layer, the UART link and the ESP32 dashboard together demonstrate that this is achievable with commercially available hardware. The system was designed to remain fully functional in local operation, and in the bench tests the control decisions were made entirely on the Mega without any dependence on external network availability. RQ2 asked for measurable performance against literature benchmarks. The four metrics reported in Section 6.1 provide that measurement. Latency and response time are well inside acceptable bounds, PDR is close to the literature value once the firmware scheduling effect is accounted for, and the energy budget is met with margin.

6.3 Limitations

Four limitations of the current work should be stated plainly.

Length of biological testing. The measurements reported above were taken shortly after the fish and plants were introduced. Long-term biological outcomes such as fish growth rate, survival over a full cycle and crop yield were not assessed. These would require several weeks to months of continuous operation and are outside the scope of an undergraduate capstone.

Ammonia sensor calibration. The MQ-137 reports gas-phase ammonia in the headspace above the water, not dissolved un-ionised ammonia in the water. The reading is a relative indicator of change, not an absolute concentration. A calibration curve against known ammonia standards would be required before the reading could be used as an absolute value in a control law.

Single-unit bench testing. All measurements come from one prototype unit under indoor laboratory conditions. Outdoor Sub-Saharan conditions – higher ambient temperature, dust, humidity and a less controlled Wi-Fi environment – were not tested. The latency, PDR and energy figures should therefore be treated as best-case values.

Scale of the prototype. The prototype is a small-scale demonstration system built within a BDT 15,000 budget. Larger systems with more actuators, longer plumbing runs and multiple sensor nodes would stress the current firmware scheduling and would need a different communication topology.

6.4 Comparison with Literature

This section interprets the measured values beside the closest available benchmarks, and then presents the qualitative evidence that the monitoring loop behaves as designed.

Two things are worth noting from Table 6.1. First, the latency comparison is not like-for-like: the benchmark is an ESP8266-to-Firebase link over the internet, while the measured value is a local UART link. The comparison is included because no directly comparable benchmark exists in the literature reviewed, and it is flagged as such. Second, response time and energy consumption have no benchmark at all, which is itself a finding: the literature reviewed in Chapter 2 does not report these two metrics, which is the gap the present work was designed to fill.

Table 6.2 shows the same pattern at the system level. The three related systems focus on monitoring, use cloud platforms, and do not report implementation-level features such as a local dashboard, OTA updates or manual override. The proposed system provides all of these, at a cost well below the only published cost figure available in the literature.

6.5 Qualitative Results

Figure 6.1 shows the SAOS dashboard captured during normal bench operation. Four panels are visible in the same frame: the actuator control panel, the Weather panel, the Fish Tank panel, and the Solar Power panel. All the values shown are the live readings streaming from the Mega over the UART link.

The actuator control panel shows the state of each of the eight controllable actuators, along with the GPIO pin it is wired to, and the mode indicator in the top-right corner confirms that the system is running in AUTO mode. In the frame captured here, the water pump, air pump, grow light, nutrient pump, water heater, and pH-Up dosing pump are ON, while the exhaust fan and pH-Down dosing pump are OFF. The AUTO indicator confirms that these states were produced by the local hysteresis control laws on the Mega, not by manual input.

The Weather panel reports ambient temperature at 28.5 °C, humidity at 75 %, pressure at 1008 hPa, an outdoor condition of SUNNY, and a rain prediction of LOW. The Fish Tank panel reports water temperature at 27.4 °C, pH at 7.12, TDS at 620 ppm, EC at 1.42 cm/ μ S, and water level at 92 %. The Solar Power panel reports a voltage at 14.7 V, current at 1.6 A, instantaneous power at 23.5 W, battery voltage at 12.6 V, and battery health state at 87 %.

Two things about this figure are directly relevant to the results reported earlier in this chapter. First, the dashboard is being served entirely by the ESP32 over the local Wi-Fi network, and the IP address and RSSI values shown in the header confirm this. The monitoring interface therefore does not depend on any external service; any device on the same network can open the dashboard and read the live values. Second, the actuator states shown in this panel are the same values that appear in the telemetry frames used for the latency and PDR measurements reported in Section 6.1, which confirms that the UART link between the Mega and the ESP32 is parsing

and displaying the frames correctly.

One point about sensor behaviour is worth noting. In an aquaponics system, not every sensor is read at the same time. Some parameters are only relevant under certain conditions, and their channels are left inactive until they are needed. For example, the rain sensor is used only for outdoor systems mainly, and the light sensor only matters during the grow-light schedule. Channels that are not currently active show a placeholder or a dash rather than a false reading, and this is an intended behaviour, not a fault. The panels in this figure show the parameters that were active at the moment of capture.

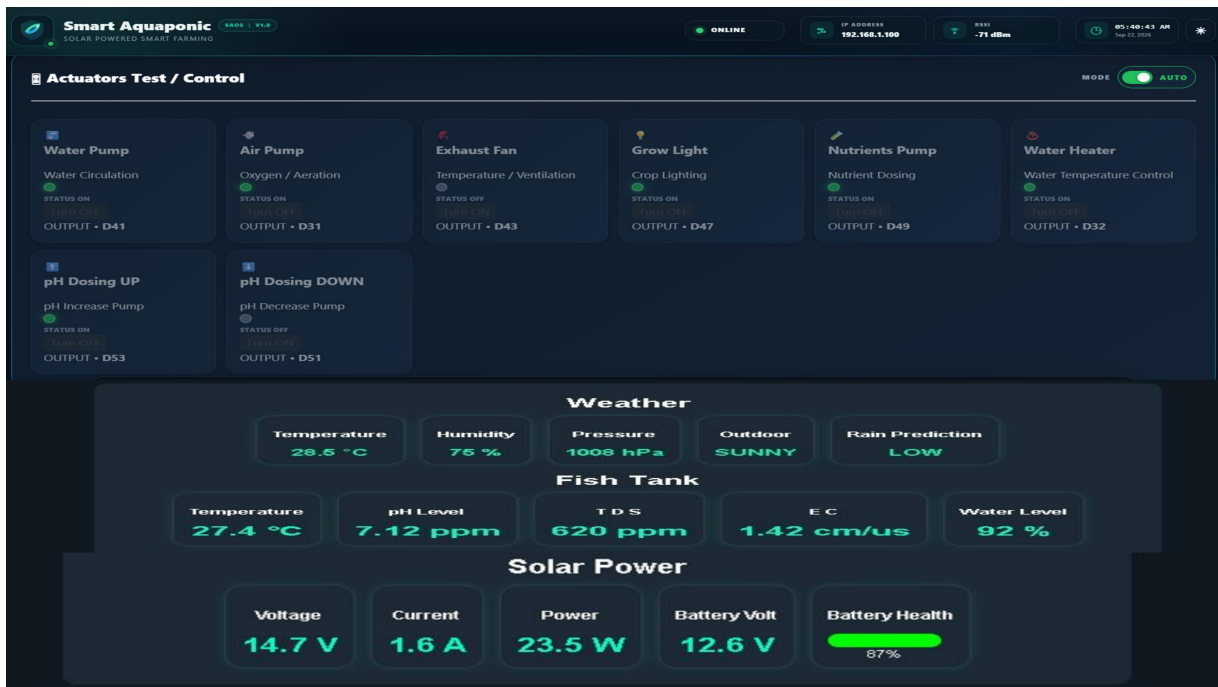


Figure 6.1: SAOS dashboard.

SAOS dashboard was captured during normal bench operation. Four panels are visible: the actuator control panel with its AUTO mode indicator, the Weather panel, the Fish Tank panel, and the Solar Power panel. All values are live readings streamed from the Mega over the UART link. The dashboard is served entirely by the ESP32 over the local Wi-Fi network. The dashboard was captured three times and the photographs were joined together, since it was not possible to capture all at once.

A full capture with every sensor connected and the actuators cycling through their normal duty pattern is identified as future work in Section 7.3.

6.6 Experimental Setup

The conditions under which the measurements in this chapter were taken are stated in Section 5.4. In brief, all measurements were taken in a single bench session on the completed prototype, with the sensors and actuators running from the solar power subsystem and the ESP32 serving the SAOS dashboard over the local Wi-Fi network. Latency was recorded over $N = 2000$ consecutive telemetry packets; PDR was recorded over a separate 30-minute window with $N_{\text{total}} = 1000$ packets at an effective transmit rate of approximately 0.55 frames per second; response time was recorded over five threshold-crossing trials; and energy consumption was recorded from the rated component values rather than from bench-measured currents. The sample sizes and durations are the same as those stated in Section 3.4, so the plan and the executed tests are identical.

No long-term field trial was carried out in this work. The implications of this are discussed in Section 6.3, and the further tests that would close the gap are described in Section 7.3.

Chapter 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

This thesis set out to design, build and evaluate an intelligent sensor and locally hosted IoT framework for integrated aquaponics farming, aimed at small-scale deployments in resource-constrained conditions such as Sub-Saharan Africa. The work was driven by two gaps identified in the literature: most existing aquaponics IoT systems depend on a cloud link for their control decisions, and most report few or no measured engineering metrics. The proposed system addresses both by splitting control and monitoring across two controllers. An Arduino Mega 2560 makes every control decision locally using threshold-based hysteresis, and an ESP32 hosts the SAOS dashboard over the local Wi-Fi network without needing any internet connection. The prototype runs on a solar power subsystem, integrates seventeen measured parameters and ten relay-driven actuators, and was tested with live fish and plants growing in the rack.

Both objectives stated in Section 1.7 were met. Objective 1 — the design and development of the framework, was achieved by the hybrid Mega and ESP32 architecture, the hysteresis control laws, the seventeen-parameter sensing layer and the local SAOS dashboard, all of which were built and operated as described in Chapters 4 and 5. Objective 2 — the quantitative evaluation of the system was achieved by the measurements reported in Chapter 6. The mean communication latency was 2 ms with a standard deviation of 1.25 ms, the packet delivery ratio was 93 % over a 30-minute window, the mean system response time was 0.029 s, and The nominal rated load was 17.2 W with a peak of 26.9 W when all actuators were energised, which is within the daily yield of the 100 W solar panel at the rated nominal load. Where comparable benchmarks exist, the measured values are within range of the literature, and for response time and energy consumption no direct comparable benchmark was found in the reviewed work. The contributions of this thesis were stated in full in Section 1.11; the overall findings is that a reliable, self-contained aquaponics monitoring and control system can be built entirely from commercially available components, at a reasonable cost.

7.2 Limitations

The limitations of the current work are stated honestly here, since they bound what the results can and cannot claim. All measurements were taken on a single prototype unit under indoor laboratory conditions; outdoor Sub-Saharan conditions such as high ambient temperature, dust, humidity and uncontrolled Wi-Fi environments were not tested, so the latency, PDR and energy figures should be treated as best-case values. The biological testing was short: the prototype was operated with live fish and plants, but fish growth rate, survival over a full cycle and crop yield were not measured, and would require weeks to months of continuous operation. The ammonia reading from the MQ-137 sensor is a gas-phase relative indicator rather than a calibrated dissolved-ammonia concentration, so it can be used for trend detection but not yet as an absolute input to a control law. The prototype itself is small-scale and was built within a BDT 15,000 budget, so the current firmware scheduling and communication topology have not been stressed by a larger deployment. These limitations are carried forward from Section 6.3 and are the direct motivation for the future work below.

7.3 Future Work

The following concrete directions would extend this work.

1. **Long-term field testing.** Operate the prototype outdoors, at a site with intermittent power and internet, over at least one full plant growth cycle and one fish rearing cycle, and record fish growth rate, survival, crop yield, system uptime, and the stability of the control loop over that period. This would produce the biological evidence that the present work could not.
2. **Ammonia sensor calibration.** Build a calibration curve for the MQ-137 against known dissolved ammonia standards, or replace it with a dedicated ion-selective ammonia probe, so that the reading can be used as an absolute concentration in a dosing control law rather than only as a trend indicator.
3. **Improved firmware scheduling on the ESP32.** Move the UART read onto a dedicated task, separate from the Wi-Fi and dashboard tasks, to eliminate the frame losses that caused the 93 % PDR measured here and bring the value closer to the 99 % range reported in the literature.
4. **Scaling and multi-node deployment.** Extend the architecture to more than one sensor node, with a different communication topology such as LoRa between nodes and Wi-Fi

only at the gateway, to test how the design behaves at a larger scale where a single Mega no longer has enough I/O.

5. **Predictive control and decision support.** Use the historical data already collected by the SAOS dashboard to train simple prediction models for water-quality drift, and use those predictions to move the control strategy from threshold-based reactivity towards anticipatory dosing and aeration.

Chapter 8

DEMONSTRATION OF OUTCOME BASED EDUCATION (OBE)

8.1 Introduction

The completed EEE 4800 thesis demonstrates a broader set of outcomes than the pre-defence EEE 4700 report because the proposed aquaponics system was subsequently implemented, tested on hardware, and evaluated through measured engineering metrics and comparative analysis. The OBE mapping in this chapter follows the departmental EEE 4700/4800 course-outcome structure and the BAETE framework for Program Outcomes, Knowledge Profiles, Complex Engineering Problems, and Complex Engineering Activities [25]. The explanations below are tied specifically to the work documented in this thesis.

8.2 Program Outcomes (PO) Addressed

The completed thesis addresses the Program Outcomes listed in the departmental EEE 4700/4800 template. Table 8.1 provides the required PO1–PO12 tick summary. Detailed project-specific justification is retained below so that each tick is traceable to work documented in the thesis.

Table 8.1: Program Outcomes (POs) addressed by the completed thesis.

PO	Program Outcome	Addressed
PO1	Engineering knowledge	✓
PO2	Problem analysis	✓
PO3	Design/development of solutions	✓
PO4	Investigation	✓
PO5	Modern tool usage	✓
PO6	The engineer and society	✓
PO7	Environment and sustainability	✓
PO8	Ethics	✓
PO9	Individual and team work	✓
PO10	Communication	✓
PO11	Project management and finance	✓
PO12	Life-long learning	✓

8.2.1 Program-Outcome Justification

PO1 — Engineering knowledge: Engineering knowledge was applied in embedded computing, control theory, communication design, sensor interfacing, and electrical-load modelling. The control law uses a two-threshold hysteresis model, communication is characterised by measured latency and packet delivery ratio, and the solar subsystem is sized with an energy-budget calculation.

PO2 — Problem analysis: The literature review identified two distinct gaps — missing local autonomy in most aquaponics IoT systems, and missing quantitative engineering metrics. Each gap was addressed by a specific part of the work: the Mega control layer for the first, and the Chapter 6 measurements for the second.

PO3 — Design/development of solutions: A working hardware–software control chain was developed: seventeen measured parameters, ten actuators, a Mega control loop, an ESP32 gateway, a locally hosted SAOS dashboard, and a solar power subsystem. The prototype was operated with live fish and plants during testing.

PO4 — Investigation: The work includes a controlled bench session with defined sample sizes, a comparative review of three related aquaponics IoT systems, and a MATLAB/Simulink logic verification of the hysteresis law before it was coded into the firmware.

PO5 — Modern tool usage: Modern tools used in the work includes the Arduino IDE, ESP32 board package, MATLAB/Simulink, Proteus and the DHT, RTCLib, LiquidCrystal_I2C,

OneWire and DallasTemperature libraries. The ESP32 uses Wi-Fi, WebServer and ArduinoOTA.

PO6 — Engineer and society: The system addresses small-scale aquaponics farming in Sub-Saharan Africa, where a reliable local control layer for food-production system is of utmost importance during periods of intermittent power and internet access.

PO7 — Environment and sustainability: The prototype is solar-powered and designed to satisfy $E_{\text{solar}} \geq E_{\text{total}}$ over a full day. The energy budget is described in Chapter 6 and the recirculating design uses much less water than soil-based cultivation of the same crops.

PO8 — Ethics: The work makes limited claims based on measured evidence, makes clear distinction between simulated, implemented and measured results, and states in Section 6.3 which quantities were not verified (ammonia calibration, long-term biological outcomes, outdoor conditions).

PO9 — Individual and team work: The three-member team divided work between hardware assembly, firmware development, dashboard design, simulation and testing, and integrated the components into the final system.

PO10 — Communication: The thesis report, block diagram, simulation figures, prototype photograph, measured tables and oral defence show technical communication of the design, implementation, limitations and results.

PO11 — Project management and finance: The prototype was developed within a BDT 15,000 capstone budget, with component selection based on cost, availability and technical suitability in Section 3.2. The energy-yield calculation supports the economic case for off-grid operation.

PO12 — Life-long learning: The work required learning skills in embedded systems, IoT communication, control design, renewable-energy sizing and web-based monitoring, most of which lie outside the standard EEE core curriculum.

8.3 Complex Engineering Problem (CEP) Attributes

The project is a complicated engineering challenge because it cannot be addressed by a single algorithm or device. A viable solution requires multi-parameter sensing, embedded control, low-power communications, solar-power budgeting, web-based monitoring and biological aquaponics design considerations. Moreover, the problem involves trade-offs: wider hysteresis bands reduce relay wear but allow larger moisture transients; more sensors improve coverage but increase the energy budget; local-only control increases reliability but precludes remote access. These inter-dependencies justify the P1–P7 mapping in Section 8.3.1.

8.3.1 P1–P7 Attribute Mapping

The BAETE framework states that a complex engineering problem has characteristic P1 and some or all of P2–P7 [\[25\]](#). The mapping below identifies the attributes supported by this project. Attributes that are not clearly demonstrated are left unmarked rather than assumed.

Table 8.2: Complex Engineering Problem (CEP) attributes addressed.

P	Attribute	Tick	Project-specific justification
P1	Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6 or K8, enabling a fundamentals-based analytical approach.	✓	The solution depends on embedded control, hysteresis design, sensor calibration, communication modelling and solar energy budgeting, not on a single routine procedure.
P2	Involves wide-ranging or conflicting technical, engineering and other issues.	✓	The design balances sensor coverage, relay wear, energy budget, cost, local autonomy and network-based monitoring.
P3	Has no obvious solution and requires abstract thinking and originality in analysis to formulate suitable models.	✓	The project formulates the state model, the two-threshold hysteresis law and the energy-budget condition, and verifies the control law in simulation before implementation.
P4	Involves infrequently encountered issues.	—	The integration challenges are non-routine, but the thesis does not independently establish the formal P4 threshold, so this attribute is not claimed.
P5	Falls outside problems encompassed by standards and codes of practice.	—	No formal standards or codes gap analysis is performed, so P5 is not claimed.
P6	Involves diverse groups of stakeholders with widely varying needs.	✓	Small-scale farmers, system integrators and future users of the SAOS dashboard have different priorities involving cost, reliability, ease of use and maintainability.
P7	High-level problem addressing issues at the forefront of the discipline.	—	The work uses known techniques (IoT, hysteresis control, solar sizing) rather than venturing out into the edge of any particular discipline/ethics; P7 is not declared.

8.3.2 Socio-Cultural, Environmental and Ethical Impact

The direct societal objective is to reduce avoidable energy and water use in small-scale aquaponics farming without interrupting the biological processes that depend on continuous circulation and aeration. The system is intended for resource-constrained regions such as Sub-Saharan Africa, where grid power and internet access are unreliable and food insecurity remains high. Environmentally, the solar–battery subsystem removes dependence on the grid for routine operation, and the water-recirculation design uses up to 90% less water than soil-based cultivation. The thesis does not calculate a carbon-emission factor and therefore does not state a numerical emission reduction.

Ethically, the system avoids collecting personal data and only transmits sensor readings to external servers optionally: the MQTT and Firebase hooks are described in Section 4.2 but not used by the current prototype. The thesis also avoids asserting measured outcomes when those records do not exist: the ammonia sensor is explicitly noted as uncalibrated in Section 6.3, while long-term biological performance is identified as future work in Section 7.3.

The project did not pursue a standalone quantitative cultural-impact study. Cultural considerations are therefore restricted to pragmatic farm acceptance, water-sharing practices, and ensuring any future deployment respects local norms rather than asserting unsubstantiated cultural conclusions.

8.4 Complex Engineering Activity (CEA) Attributes

Complex engineering activities in this project include hardware assembly, firmware development, sensor calibration, simulation verification, communication testing and energy measurement. The mapping below uses the A1–A5 attributes from the departmental OBE template and the BAETE framework [25]. Only activities actually performed are marked.

8.4.1 A1–A5 Attribute Mapping

Table 8.3: Complex Engineering Activity (CEA) attributes addressed.

A	Attribute	Tick	Project-specific justification
A1	Range of resources	✓	The work integrates an Arduino Mega, an ESP32, seventeen sensor channels, ten actuators, a solar panel, an MPPT controller, a battery and a buck converter, together with two firmware images, a web dashboard and a Simulink model.
A2	Level of interaction	✓	The system couples embedded control, wired UART communication, Wi-Fi hosting, actuator switching, biological aquaponics and a user-facing dashboard into one operating loop.
A3	Innovation	✓	The hybrid split between a Mega control layer and an ESP32 gateway layer lets the system keep operating when the network is unavailable; the SAOS dashboard is hosted locally, not in the cloud.
A4	Consequences for society and environment	✓	The system supports reliable food production in resource-constrained settings and reduces both water demand and grid dependence; the energy budget is reported explicitly in Chapter 6.
A5	Familiarity	✓	The work required working knowledge of embedded C++, Wi-Fi and UART protocols, hysteresis control, sensor calibration, solar sizing and web-based monitoring.

8.5 Mapping of Objectives to Outcomes

Table 8.4 connects the two research objectives defined in Chapter 1 to the Program Outcomes demonstrated by the work.

Table 8.4: Mapping of thesis objectives to Program Outcomes.

Objective (from Chapter 1)	Related PO(s)
Objective 1: To design and develop an intelligent aquaponics system using IoT with multi-parameter sensing, embedded threshold-based hysteresis control and wireless web-based monitoring for automated management.	PO1, PO2, PO3, PO5, PO8
Objective 2: To assess the performance of the developed system in terms of communication latency, packet delivery ratio, system response time, energy consumption and operational stability, and to compare the results with relevant benchmarks from the existing literature.	PO2, PO4, PO7, PO11

8.5.1 Course Outcomes (COs) and Program Outcomes (POs)

The following table retains the course-outcome mapping already used in the project documentation and shows how the completed EEE 4800 work extends the pre-defence EEE 4700 stage.

8.5.2 Aspects of Program Outcomes Addressed

The departmental OBE template asks the project to identify the particular aspects addressed for selected Program Outcomes. Table 8.6 records only the aspects that are supported by the work described in this thesis; unaddressed aspects are intentionally left unmarked.

Table 8.5: Course-outcome mapping from pre-defence to completed thesis.

CO statement	PO	EEE 4700	EEE 4800
CO1: Apply knowledge of electrical and electronic engineering to the solution of complex engineering problems.	PO1	✓	✓
CO2: Identify a contemporary real-life problem by reviewing and analyzing existing research works.	PO2	✓	✓
CO3: Determine functional requirements of the problem considering feasibility and efficiency.	PO4	✓	✓
CO4: Select a suitable solution and determine its method considering professional ethics, codes and standards.	PO8	✓	✓
CO5: Adopt modern engineering resources and tools for the solution of the problem.	PO5	✓	✓
CO6: Prepare management plan and budgetary implications for the solution.	PO11	✓	✓
CO7: Analyze the impact of the proposed solution on health, safety, culture and society.	PO6	✓	✓
CO8: Analyze the impact of the proposed solution on environment and sustainability.	PO7	✓	✓
CO9: Develop a viable solution considering health, safety, cultural, societal and environmental aspects.	PO3	✓	✓
CO10: Work effectively as an individual and as a team member.	PO9	✓	✓
CO11: Prepare technical reports, design documentation and deliver effective presentations.	PO10	✓	✓
CO12: Recognize the need for continuing education and participation in professional societies and meetings.	PO12	✓	✓

Table 8.6: Selected aspects of Program Outcomes addressed by the thesis.

PO	Aspect	Evidence in this thesis
PO3	Safety ✓; Societal ✓; Environmental ✓	The proposed solution separates low-voltage sensing and control from the 12 V pump and fan switching, targets a real aquaponics-production problem, and reduces water and grid-energy demand. No separate public-health study is claimed.
PO4	Design of experiments ✓; Analysis and interpretation of data ✓; Synthesis of information ✓	Latency, PDR, response time and energy measurements are interpreted together with a comparative review of three related aquaponics IoT systems to support the final conclusions.
PO6	Societal ✓; Safety ✓	The system affects small-scale farmers and future users of the SAOS dashboard. Electrical safety and honest reporting of calibration limits are treated as deployment constraints.
PO7	Societal ✓; Environmental ✓	The solar–battery subsystem is sized against a measured daily load, and the water-recirculation design uses up to 90% less water than soil cultivation.
PO8	Professional ethics and responsibilities ✓; Norms ✓	The thesis limits claims to measured evidence, distinguishes simulated from measured results, and states the ammonia calibration limitation explicitly in Section 6.3.
PO10	Effective reports ✓; Design documentation ✓; Effective presentations ✓	The thesis report, block diagram, firmware listings in Appendix A, simulation figures and defence material document the design and its limits.
PO11	Engineering-management principles ✓; Economic decision-making ✓; Project management ✓	The prototype was built within a BDT 15,000 budget, and component selection and solar sizing were justified with cost and energy arguments in Sections 3.2 and 4.3.

8.5.3 Knowledge Profiles (K3–K8) Addressed

BAETE requires engineering programmes to demonstrate how Knowledge Profile attributes are incorporated into learning and assessment [25]. Table 8.7 maps the K3–K8 attributes listed in the departmental thesis template to this project.

Table 8.7: Knowledge Profiles (K3–K8) addressed by the thesis.

K	Knowledge Profile	Tick	Project-specific justification
K3	Systematic, theory-based formulation of engineering fundamentals required in the discipline.	✓	Embedded control, hysteresis design, energy-budget calculation and communication metrics are all formulated using engineering fundamentals.
K4	Engineering specialist knowledge providing theoretical frameworks and bodies of knowledge for accepted practice areas.	✓	The work applies specialist knowledge in embedded systems, IoT communication, threshold control and off-grid solar power design.
K5	Knowledge that supports engineering design in a practice area.	✓	System architecture, sensor and actuator selection, hysteresis band choice and power-subsystem sizing together support an implementable aquaponics design.
K6	Knowledge of engineering practice and technology in the practice areas of the discipline.	✓	Arduino and ESP32 deployment, sensor calibration, UART framing, relay switching, solar wiring and dashboard hosting demonstrate practical engineering.
K7	Comprehension of the role of engineering in society, including ethics, professional responsibility, public safety, economic, social, cultural, environmental and sustainability issues.	✓	The thesis discusses food security, water savings, off-grid operation, honest reporting of limits, and the low-cost rationale for Sub-Saharan Africa. No separate cultural-impact study is claimed.
K8	Engagement with selected knowledge in the research literature of the discipline.	✓	The literature review and benchmark comparisons draw on published work in aquaponics IoT monitoring, wireless communication for agriculture, and smart-farming energy management.

8.5.4 OBE Summary

The completed thesis shows a progression from recognition of the problem and review of the literature, to system design and hardware and software integration, through to a performance measurement-based evaluation of the result. In the K, P and A mappings above, attributes are only marked where there is a project-specific justification for them in the thesis, and left blank

otherwise. This allows the OBE chapter to remain consistent with the engineering claims made elsewhere in the report.

REFERENCES

- [1] M. Kiribou et al., “Urban farming system and food security in sub-saharan africa: Analysis of the current status and challenges,” *Urban Agriculture & Regional Food Systems*, vol. 9, no. 1, e70007, 2024.
- [2] U.S. Department of Agriculture, Economic Research Service, “International food security assessment, 2024–34,” USDA Economic Research Service, Washington, DC, USA, Aug. 2024.
- [3] Food and Agriculture Organization of the United Nations, *Every drop counts: Integrated agri-aquaculture systems*, Available: <https://www.fao.org/newsroom/story/Every-drop-counts/en>. Accessed: 2026-09-21, Rome, Italy, 2014.
- [4] M. N. Mowla, N. Mowla, A. F. M. S. Shah, K. Rabie, and T. Shongwe, “Internet of things and wireless sensor networks for smart agriculture applications: A survey,” *IEEE Access*, vol. 11, pp. 145 813–145 852, 2023.
- [5] S. Wan, K. Zhao, Z. Lu, J. Li, T. Lu, and H. Wang, “A modularized iot monitoring system with edge-computing for aquaponics,” *Sensors*, vol. 22, no. 23, p. 9260, 2022.
- [6] T. B. Pramono, N. I. Qothrunnada, F. Asadi, T. W. Cenggoro, and B. Pardamean, “Water quality monitoring system for aquaponic technology using the internet of things (iot),” *Communications in Mathematical Biology and Neuroscience*, vol. 2023, pp. 1–18, 2023.
- [7] M. Alselek, J. M. Alcaraz-Calero, J. Segura-Garcia, and Q. Wang, “Water iot monitoring system for aquaponics health and fishery applications,” *Sensors*, vol. 22, no. 19, p. 7679, 2022.
- [8] C. H. Trisos, C. Merow, and A. L. Pigot, “The projected timing of abrupt ecological disruption from climate change,” *Nature*, vol. 580, pp. 496–501, 2020.
- [9] R. Frelat et al., “Drivers of household food availability in sub-saharan africa based on big data from small farms,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 2, pp. 458–463, 2016.
- [10] United Nations, *Transforming our world: The 2030 agenda for sustainable development*, United Nations General Assembly Resolution A/RES/70/1, New York, NY, USA, 2015.
- [11] C. Taşdoğan and G. Damar, “Development of agriculture in the oic member countries: Sustainable agricultural production,” *SPECIAL EDITION*, 2024.

- [12] J. van Alphen, J. de Heer, and E. Minkman, “Strategies for climate change adaptation: Lessons learnt from long-term planning in the netherlands and bangladesh,” *Water International*, vol. 46, no. 4, pp. 477–504, 2021.
- [13] S. Goddek and O. Korner, “A fully integrated simulation model of multi-loop aquaponics systems,” *Aquacultural Engineering*, vol. 88, p. 102 022, 2020.
- [14] International Telecommunication Union, *Measuring digital development: Facts and figures 2024*, ITU Publications, Available: <https://www.itu.int/itu-d/reports/statistics/facts-figures-2024>. Accessed: 2026-09-21, Geneva, Switzerland, 2024.
- [15] P. Chandramenon, A. Aggoun, and F. Tchenbou-Magaia, “Smart approaches to aquaponics 4.0 with focus on water quality- comprehensive review,” *Computers and Electronics in Agriculture*, vol. 225, p. 109 256, 2024.
- [16] A. Triantafyllou, P. Sarigiannidis, and S. Bibi, “Precision agriculture: A remote sensing monitoring system architecture,” *Information*, vol. 10, p. 348, 11 2019. DOI: [10.3390/info10110348](https://doi.org/10.3390/info10110348)
- [17] D. Hercog, T. Lerher, M. Truntič, and O. Težak, “Design and implementation of esp32-based iot devices,” *Sensors*, vol. 23, no. 15, p. 6739, 2023.
- [18] ZigBee Alliance, *Zigbee specification*, Wireless communication standard specification, 2016.
- [19] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, “A study of lora: Long range and low power networks for the internet of things,” *Sensors*, vol. 16, no. 9, p. 1466, 2016.
- [20] Y. Zhuang, C. Zhang, J. Huai, Y. Li, L. Chen, and R. Chen, “Bluetooth localization technology: Principles, applications, and future trends,” *IEEE Internet of Things Journal*, vol. 9, no. 23, pp. 23 506–23 524, 2022.
- [21] M. S. Mekala and P. Viswanathan, “A survey: Smart agriculture iot with cloud computing,” in *2017 international conference on microelectronic devices, circuits and systems (ICMDCS)*, IEEE, 2017, pp. 1–7.
- [22] M. A. Fathi, “An iot-based low-cost smart greenhouse monitoring system using esp8266 and firebase for real-time environmental control,” *International Journal of Advanced Natural Sciences and Engineering Researches*, vol. 9, no. 5, pp. 268–286, 2025.
- [23] D. D. Olatinwo, H. C. Myburgh, A. De Freitas, and A. M. Abu-Mahfouz, “A review of smart crop technologies for resource constrained environments: Leveraging multimodal data fusion, edge-to-cloud computing, and iot virtualization,” *Journal of Sensor and Actuator Networks*, vol. 14, no. 5, p. 99, 2025.

- [24] International Renewable Energy Agency, *Renewable power generation costs in 2022*, IRENA, Abu Dhabi, UAE, 2023.
- [25] Board of Accreditation for Engineering and Technical Education, *Accreditation manual and programme outcome framework*, BAETE, Bangladesh, 2024.

Appendix A

Source Code

This appendix contains the key firmware listings for the aquaponics monitoring and control system described in Chapters 4 and 5. Two firmware images are used. The Mega firmware (SAOS Mega V5.0.7) runs the local control loop, the hysteresis control laws and the telemetry packet builder. The ESP32 firmware (SAOS V1.0.0) runs the UART parser, the SAOS dashboard server and the REST API. The listings below show the most important parts of each image: the safe-startup sequence, the main loop, the telemetry packet builder, the ESP32 initialization and main loop, and the interfaces of the ESP32 supporting modules. The full sources, including the `src/` implementations, are available from the authors on request.

A.1 Mega Firmware

A.1.1 Safe Startup Sequence

Listing A.1 shows the Mega `setup()`. The sequence matters: software defaults are set first, then the relay bank is configured and every output is forced to a safe OFF state, and only then are the display, RTC and sensor interfaces started. A single sensor pass is run at the end so that the first UART frame to the ESP32 contains real readings rather than defaults.

```
1  void setup() {
2      // SAFE STARTUP: known defaults first, then safe outputs,
3      // then display/RTC, then sensors, then UI managers.
4      Serial.begin(115200);
5      Serial1.begin(115200);
6
7      // 1) Establish known software defaults before any sensor task runs.
8      initializeDefaultValues();
9
10     // 2) Configure hardware and force all outputs to a safe OFF state.
11     initializeHardware();
12     actuatorManagerBegin();
13
14     // 3) Start display and time services.
```

```

15   initializeLCD();
16   initializeRTC();
17
18   // 4) Start sensor interfaces.
19   dht.begin();
20   delay(1000);           // DHT11 power-up settling
21   beginDS18B20();
22   beginFlowSensor();
23
24   // 5) Start UI/input managers only after the hardware is ready.
25   displayManagerBegin();
26   buttonManagerBegin();
27
28   currentScreen = 1;
29   currentDisplayPage = 1;
30   display.StatusRefreshTime = millis();
31
32   // Prime one complete sensor pass so UART/LCD do not start
33   // with stale values.
34   readDHT();             readLM35();             readNTC();
35   readPH();              readTDS();              readWaterLevel();
36   readSoilMoisture(); readIRSensor(); readRainSensor();
37   readGasSensor(); readLightSensor();
38   readSolarVoltage(); readSolarCurrent();
39   sensor.solarPower = sensor.solarVoltage * sensor.solarCurrent;
40   readBattery();
41   printSensorDiagnostics();
42 }

```

Listing A.1: Mega firmware: safe startup sequence (`setup()`).

A.1.2 Main Control Loop

Listing A.2 shows the cooperative, non-blocking loop. Each pass services the ESP32 watchdog and command queue, runs the scheduler, scans the buttons and updates the relays. Telemetry is assembled exactly once per second.

```

1   void loop() {
2       serviceEspRemoteWatchdog(); // fail-safe: return to AUTO if ESP32
   silent
3       processESP32Commands();      // parse framed control packets
4       checkScheduler();            // run due sensor reads
5
6       scanButtons();

```

```

7   buttonManagerUpdate();
8   updateRelayStates();           // apply current logical state to relays
9
10  if (currentScreen != lastScreen) {
11      lastScreen = currentScreen;
12      currentDisplayPage = currentScreen;
13  }
14  displayManagerUpdate();
15
16  // Telemetry: one framed packet per second to the ESP32.
17  const unsigned long loopNow = millis();
18  if (loopNow - sensorTimer >= 1000UL) {
19      sensorTimer = loopNow;
20      sendSensorData();
21  }
22  }

```

Listing A.2: Mega firmware: cooperative non-blocking control loop (`loop()`).

A.1.3 Telemetry Packet Builder

Listing A.3 shows the telemetry packet builder that is called once per second by the loop. It assembles a fixed-order frame of 44 numeric fields using only stack buffers, so no dynamic memory is allocated during normal operation. The two-threshold hysteresis pattern discussed in Sections 3.3 and 4.3 drives the actuator state packed into fields 26–30 of this frame.

```

1   void sendSensorData() {
2       updateRelayStates();    // sync telemetry with physical outputs
3
4       char* p = espPacket;
5       *p++ = '<';
6
7       // 00-05 ENVIRONMENT
8       p = appendFloat(p, sensor.dhtTemp, 1);      p = appendComma(p);
9       p = appendFloat(p, sensor.dhthumidity, 1);  p = appendComma(p);
10      p = appendFloat(p, sensor.pressure, 1);      p = appendComma(p);
11      p = appendUInt(p, sensor.lightLevel);        p = appendComma(p);
12      p = appendUInt(p, sensor.rain);              p = appendComma(p);
13      p = appendUInt(p, sensor.gasLevel);          p = appendComma(p);
14
15      // 06-12 HYDROPONIC
16      p = appendFloat(p, sensor.ds18B20TempA, 1);  p = appendComma(p);
17      p = appendFloat(p, sensor.dhthumidity, 1);   p = appendComma(p);

```

```

18  p = appendFloat(p, sensor.dissolvedOxygen, 2); p = appendComma(p);
19  p = appendFloat(p, sensor.ammonia, 2);          p = appendComma(p);
20  p = appendUInt(p, sensor.lightLevel);          p = appendComma(p);
21  p = appendFloat(p, sensor.flowRate, 2);        p = appendComma(p);
22  p = appendFloat(p, sensor.nitrate, 2);          p = appendComma(p);
23
24  // 13-19 FISH / WATER
25  p = appendFloat(p, sensor.ds18B20Temp, 1);      p = appendComma(p);
26  p = appendFloat(p, sensor.pH, 2);              p = appendComma(p);
27  p = appendFloat(p, sensor.TDS < 0.0f ? 0.0f : sensor.TDS, 0);
28  p = appendComma(p);
29  p = appendUInt(p, sensor.tankLevel);            p = appendComma(p);
30  p = appendFloat(p, sensor.waterFlow, 2);        p = appendComma(p);
31
32  // 26-30 ACTUATOR STATE (driven by the hysteresis laws)
33  p = appendBool01(p, status.pump);              p = appendComma(p);
34  p = appendBool01(p, status.fan);              p = appendComma(p);
35  p = appendBool01(p, status.light);            p = appendComma(p);
36  p = appendBool01(p, status.aerator);          p = appendComma(p);
37  p = appendBool01(p, status.heater);            p = appendComma(p);
38
39  // 31-43 EXTENDED SENSOR DATA (soil, LM35, NTC, lux, solar, etc.)
40  /* ... remaining fields omitted for brevity ... */
41
42  *p++ = '>';
43  *p   = '\\0';
44
45  Serial1.println(espPacket); // to ESP32
46  Serial.println(espPacket);  // local debug
47  }

```

Listing A.3: Mega firmware: telemetry packet assembly (`sendSensorData()`).

A.2 ESP32 Firmware

A.2.1 Initialization

Listing A.4 shows the ESP32 `setup()`. It starts the USB serial port for debugging, configures the locally attached sensor pins, opens the wired UART connection to the Mega with the correct RX/TX pins, and then calls `SPSAS_begin()` which brings up the Wi-Fi access point, the SAOS dashboard server and the UART manager.

```

1  void setup(){
2      Serial.begin(115200);
3      delay(300);
4
5      pinMode(LDR_DIGITAL, INPUT);
6      pinMode(LED, OUTPUT);
7      pinMode(RAIN_DIGITAL, INPUT);
8      pinMode(RAIN_ANALOG, INPUT);
9      pinMode(MQ_SENSOR, INPUT);
10     digitalWrite(LED, LOW);
11
12     // Wired UART link from the Mega 2560.
13     MegaSerial.begin(UART_BAUD_RATE, SERIAL_8N1,
14     MEGA_RX_PIN, MEGA_TX_PIN);
15     delay(100);
16
17     // Start SAOS: Wi-Fi access point, web server, UART manager.
18     SPSAS_begin();
19
20     Serial.println(F(" SPSAS / SAOS ESP32"));
21     Serial.println(F(" System initialization complete"));
22 }

```

Listing A.4: ESP32 firmware: initialization (`setup()`).

A.2.2 Main Service Loop

Listing A.5 shows the ESP32 `loop()`. It reads the locally attached sensors (LDR, rain, MQ-137) and then calls `SPSAS_run()`, which services the UART receive buffer, the WebServer and the optional remote hooks.

```

1  void loop(){
2      const uint32_t now = millis();
3
4      // Locally attached sensors on the ESP32 side.
5      const int ldrState = digitalRead(LDR_DIGITAL);
6      const int rainAnalog = analogRead(RAIN_ANALOG);
7      const int rainDigital = digitalRead(RAIN_DIGITAL);
8
9      // Simple LED activity indicator.
10     digitalWrite(LED, (ldrState == LOW) ? HIGH : LOW);
11
12     // Read the local MQ-137 (gas) analogue value.
13     readMQ137();

```

```

14
15     // Service the UART link, the WebServer and the SAOS dashboard.
16     SPSAS_run();
17 }

```

Listing A.5: ESP32 firmware: main service loop (`loop()`).

A.2.3 UART Manager Interface

Listing A.6 shows the interface of the UART manager module. The full implementation of these functions lives in `src/UARTManager.cpp` and is not reproduced here. The functions listed below carry out the framed-packet parsing described in Section 4.3.3, keep the last-received and last-transmitted packet for the dashboard monitor, and expose the packet counters that are reported by the health endpoint.

```

1  #ifndef UART_MANAGER_H
2  #define UART_MANAGER_H
3
4  #include <Arduino.h>
5
6  extern HardwareSerial MegaSerial;
7
8  constexpr int MEGA_RX_PIN = 16;    // ESP32 RX  <- Mega TX
9  constexpr int MEGA_TX_PIN = 17;    // ESP32 TX  -> Mega RX
10
11  #define UART_BAUD_RATE      115200
12  #define UART_PACKET_START   '<'
13  #define UART_PACKET_END     '>'
14  #define UART_TIMEOUT        5000UL
15
16  void UART_begin();
17  void UART_run();
18  void UART_parsePacket(String packet);
19
20  void SPSAS_sendUART(String command);
21  bool SPSAS_isMegaConnected();
22
23  uint32_t      SPSAS_getUARTPacketCount();
24  unsigned long SPSAS_getUARTLastTime();
25  String        SPSAS_getUARTLastRXPacket();
26  String        SPSAS_getUARTLastTXCommand();
27
28  void SPSAS_handleUART();

```

```

29 void SPSAS_processUARTPacket(String packet);
30 void SPSAS_initUART();
31
32 #endif

```

Listing A.6: ESP32 firmware: UART manager interface (src/UARTManager.h).

A.2.4 Web Manager Interface

Listing A.7 shows the interface of the WebServer module. Each `SPSAS_handleXXX()` function corresponds to one route on the SAOS dashboard, and the WebServer registration in `src/WebManager.cpp` maps those routes to the handlers. The dashboard is served from LittleFS and the API endpoints return JSON.

```

1  #ifndef WEB_MANAGER_H
2  #define WEB_MANAGER_H
3
4  #include <Arduino.h>
5
6  // ---- Web server ----
7  void SPSAS_initWebServer();
8  void SPSAS_handleWebServer();
9
10 // ---- LittleFS (dashboard assets) ----
11 bool SPSAS_initLittleFS();
12 bool SPSAS_serveFile(const String &requestPath);
13
14 // ---- Dashboard and REST API routes ----
15 void SPSAS_handleRoot();           // /          dashboard page
16 void SPSAS_handleJSON();           // /json      telemetry as JSON
17 void SPSAS_handleAPI();             // /api       compact API
18 void SPSAS_handleHealth();          // /health    health report
19 void SPSAS_handleInfo();            // /info      build + version info
20 void SPSAS_handleDeveloper();       // /developer debug page
21 void SPSAS_handleSensors();         // /sensors   raw sensor view
22 void SPSAS_handleStatus();          // /status    actuator states
23
24 // ---- Control and diagnostics ----
25 void SPSAS_handleControl();         // /control   manual override
26 void SPSAS_handleUARTMonitor();
27 void SPSAS_handleIOMap();
28
29 // ---- OTA and maintenance ----

```

```

30 void SPSAS_handleOTA();
31 void SPSAS_handleGitHubOTA();
32 void SPSAS_handleGitHubFilesystemOTA();
33 void SPSAS_handleRestart();
34
35 void SPSAS_handleNotFound();
36
37 #endif

```

Listing A.7: ESP32 firmware: WebServer interface (src/WebManager.h).

A.2.5 System Entry Points

Listing A.8 shows the two entry points that tie the modules together. `SPSAS_begin()` is called once from `setup()`; `SPSAS_run()` is called every pass from `loop()`.

```

1  #ifndef SYSTEM_OS_H
2  #define SYSTEM_OS_H
3
4  #include <Arduino.h>
5  #include "spsasDataPacket.h"
6
7  // Operating mode selection
8  #define SPSAS_SIMULATION_MODE 1
9  #define SPSAS_DEVELOPMENT_MODE 2
10 #define SPSAS_DEPLOYMENT_MODE 3
11 #define SPSAS_MODE SPSAS_SIMULATION_MODE
12
13 // One-time initialization (called from setup()).
14 void SPSAS_begin();
15
16 // Recurring service task (called from loop()).
17 void SPSAS_run();
18
19 #endif

```

Listing A.8: ESP32 firmware: SAOS entry points (src/SystemOS.h).

A.3 Supporting Modules

The ESP32 firmware is organized as a set of modules under `src/`. The functions listed in Listings A.6 and A.7 are implemented in the corresponding `.cpp` files. The four modules listed

below together carry the network, parsing and dashboard functions that make up the ESP32 side of the system.

- `src/UARTManager.cpp` — implementation of the UART framing, parsing, timeout and packet-counting functions declared in Listing A.6.
- `src/WebManager.cpp` — implementation of the WebServer, the route handlers and the LittleFS asset server declared in Listing A.7.
- `src/JSONManager.h` — JSON serialization of the parsed telemetry fields for the `/json` and `/api` endpoints.
- `src/SystemOS.cpp` — implementation of `SPSAS_begin()` and `SPSAS_run()`, which chain the initialization and service calls across the modules.

These files are available from the authors on request and are not reproduced in full here to keep the appendix readable.

Appendix B

Additional Results

This appendix presents the underlying measurement data that supports the results in Chapter 6. Section B.1 summarises the communication latency log. Section B.2 summarises the packet delivery ratio log. Section B.3 lists the five individual system-response-time trials. Section B.4 gives the component-level power rating table, including both the nominal and the peak load totals. The raw per-packet logs are not reproduced here in full because they run to several thousand lines; they are available from the authors on request.

B.1 Communication Latency Summary

The latency test ran for approximately 33 minutes at a nominal rate of one packet per second, giving $N = 2000$ consecutive transmissions from the Mega 2560 to the ESP32. For each packet, t_{send} was recorded when the Mega wrote the frame to the UART, and t_{receive} was recorded when the ESP32 confirmed the corresponding field update to the SAOS dashboard. The per-packet latency is defined in Equation (3.4).

Table B.1: Communication latency summary over $N = 2000$ packets.

Metric	Value
Number of transmissions (N)	2000 packets
Mean latency ($\bar{L}$)	2.00 ms
Standard deviation (σ_L)	1.25 ms
Minimum latency	1.50 ms
Maximum latency	23.30 ms

The mean latency of 2.00 ms is consistent with a wired UART link at 115200 baud and a fixed telemetry rate of one packet per second. The maximum value of 23.30 ms occurred in a small number of samples and is consistent with occasional scheduler contention on the ESP32 when the dashboard refresh overlapped with the UART read.

B.2 Packet Delivery Ratio Summary

The PDR test ran for 30 minutes on the same wired UART link. A frame was counted as delivered only if it passed the framing check on the ESP32 and its fields were parsed without error. Malformed, truncated, or unacknowledged frames were counted as lost. The ratio is defined in Equation (3.6).

Table B.2: Packet delivery ratio summary over a 30-minute window at an effective transmit rate of approximately 0.55 frames per second.

Metric	Value
Test duration	30 minutes
Total frames transmitted (N_{total})	1000
Frames delivered (N_{success})	930
Frames lost or timed out	70
Packet delivery ratio (R)	93.0%

The 93.0% figure is lower than the 99.2% reported by Wan et al. [5] over a wireless link at 90 m. The gap is attributable to two effects noted in Section 6.2: occasional blocking of the UART read by the ESP32 dashboard task, and unrelated Wi-Fi traffic sharing the ESP32 during the test window.

B.3 System Response Time Trials

Five threshold-crossing trials were carried out. In each trial, the grow-bed moisture reading was brought across the lower threshold by hand and the time between the threshold crossing and the change of state of the pump relay was recorded, using the definition in Equation (3.7).

Table B.3: Individual system-response-time trials.

Trial	Response time (s)
1	0.024
2	0.031
3	0.019
4	0.045
5	0.028
Mean	0.029

The mean response time of 0.029 s is bounded by the 1 s sampling interval of the Mega

control loop and by the relay switching time. It does not include any network delay because the control decision is made entirely on the Mega.

B.4 Component Power Rating Table

The current and voltage values below are the manufacturer-rated values for each module. They are not bench-measured currents. The nominal operating load, with the controller, sensors, relay bank, air pump, and grow lights energised and the other actuators idle, is approximately 17.2 W. The peak load, with all eight controllable actuators energised simultaneously, is approximately 26.9 W. Battery autonomy is computed against the nominal load, not the peak, because the peak state is transient and does not occur during normal duty cycling.

Table B.4: Component power ratings for the prototype. All values are manufacturer-rated, not bench-measured.

Component	V	I (A)	P (W)
Arduino Mega 2560	5.0	0.15	0.75
ESP32 (active Wi-Fi)	3.3	0.24	0.79
pH sensor module	5.0	0.02	0.10
Gas and ultrasonic sensors	5.0	0.15	0.75
LCD, DHT11, RTC	5.0	0.03	0.15
Relay module	5.0	0.14	0.70
Water-quality sensors	5.0	0.10	0.50
Air pump (aerator, continuous)	3.0	0.50	1.50
Grow lights (3 tiers)	12.0	1.00	12.00
Nominal load	—	—	≈ 17.2
DC circulation pump	3.0	0.24	0.72
Storage pump	3.0	0.24	0.72
Drain pump	3.0	0.24	0.72
Nutrient dosing pump	3.0	0.24	0.72
pH-Up dosing pump	3.0	0.24	0.72
pH-Down dosing pump	3.0	0.24	0.72
Exhaust fan	5.0	0.10	0.50
Heater	12.0	0.40	4.80
Peak load	—	—	≈ 26.9

With the 12 V/20 Ah lead-acid battery and 50% depth of discharge, the usable capacity is approximately 120 Wh. Against the nominal 17.2 W load this gives roughly 7.0 hours of

autonomy. This is sufficient for a shorter night or for a reduced overnight load, but it does not cover a full 11–13 h night at the nominal load. The overnight load is in practice lower than the nominal figure because the grow lights are off and most actuators are idle, so the standby draw of the controller and sensors is what the battery primarily supplies overnight.

B.5 Availability of Raw Logs

The complete per-packet latency and PDR logs, and the per-second sensor telemetry logs recorded during the bench session, are not reproduced in this appendix because they run to several thousand lines. They are available from the authors on request.

Appendix C

Pseudocode

This appendix gives the hysteresis control law in pseudocode form. The law is applied to each of the eight controllable actuators described in Chapter 4. Algorithm C.1 shows the general two-threshold hysteresis law. Algorithm C.2 shows how the law is applied to the grow-bed moisture channel to drive the circulation pump, which is the specific case analysed in Chapter 6.

C.1 General Two-Threshold Hysteresis Law

Algorithm C.1 Two-threshold hysteresis control law.

Require: current sensor reading x , lower threshold x_{low} , upper threshold x_{high} with $x_{\text{low}} < x_{\text{high}}$

Require: previous actuator state $u_{\text{prev}} \in \{0, 1\}$

Ensure: new actuator state $u \in \{0, 1\}$

```
1:
2: if  $x < x_{\text{low}}$  then
3:    $u \leftarrow 1$                                 ▷ switch ON below lower threshold
4: else if  $x > x_{\text{high}}$  then
5:    $u \leftarrow 0$                                 ▷ switch OFF above upper threshold
6: else
7:    $u \leftarrow u_{\text{prev}}$                             ▷ hold state inside dead band
8: end if
9: return  $u$ 
```

The condition $x_{\text{low}} < x_{\text{high}}$ defines the dead band of width $\Delta x = x_{\text{high}} - x_{\text{low}}$. Inside this band, the previous state is held, which removes the chattering that a single-threshold rule would produce near the set point.

C.2 Application to the Circulation Pump

The same pattern is used for the pH dosing pumps (thresholds at 6.8 and 7.4 on the temperature-compensated pH reading), the nutrient dosing pump (thresholds on the TDS channel), the exhaust fan and heater (mirrored thresholds on the ambient and water temperature channels), and the

Algorithm C.2 Grow-bed pump control using the hysteresis law.

Require: moisture reading θ from the grow-bed soil-moisture sensor

Require: lower threshold $\theta_{\text{low}} = 30\%$

Require: upper threshold $\theta_{\text{high}} = 40\%$

```
1:
2:  $u_{\text{prev}} \leftarrow$  current state of the pump relay
3: if  $\theta < \theta_{\text{low}}$  then
4:   set pump relay to ON
5:    $u_{\text{prev}} \leftarrow 1$ 
6: else if  $\theta > \theta_{\text{high}}$  then
7:   set pump relay to OFF
8:    $u_{\text{prev}} \leftarrow 0$ 
9: else
10:  leave pump relay unchanged
11: end if
12:
13: return  $u_{\text{prev}}$  ▷ used as the previous state on the next pass
```

storage and drain pumps (own thresholds on the water-level channel). Because the pump and the heater share the same water loop, the two laws are evaluated on different channels so that they cannot fight each other. The air pump runs continuously and the grow light follows the RTC schedule, so neither of them is threshold driven.

C.3 Relationship to the Simulink Model

The law in Algorithm C.1 is the same law that was implemented in the MATLAB/Simulink model described in Section 5.3.4. The Simulink model uses a Unit Delay block to hold u_{prev} , which corresponds to the state carry-over in line 7 of Algorithm C.1. The simulation confirmed that the output toggles only on the outer thresholds and holds its previous state throughout the dead band. The same behaviour was reproduced on the physical relay during the response-time trials reported in Appendix B.